

DSv4 CSA/HCA 投机预取设计说明

本文是当前 DSv4 + LMCache + NVMe KV 预取工作的唯一主设计文档。以后 compact 之后先读本文，再读运行注意事项：
F:\LMCache\docs\design\v1\dsv4_csa_hca_prefetch_runbook.md。以后请在每个实验下面标明使用的代码git版本

2026-05-30 关键纠偏：CPU pinned 只能是临时 I/O buffer

HCA overlap 的目标路径是让 **SSD/NVMe 读与 MoE/FFN/A2A 窗口重叠**，最终数据必须进入 GPU/HBM staging 或目标 KV cache。可以分配较大的 CPU pinned memory 作为临时 I/O bounce buffer，但它不能成为 cache：

- 不参与 LMCache hit 语义。
- 不作为 resident set。
- 不保护、不复用具体 KV 内容。
- 一次 I/O 完成后立刻把 payload 送入 HBM staging/目标 KV，buffer 归还给 slab。

错误路径是把 CPU pinned/DRAM 当作长期 cache 或命中来源：

```
SSD -> CPU pinned cache/resident set -> 后续 attention 按 CPU resident 命中
```

可接受的临时 buffer 路径：

```
SSD/NVMe -> CPU pinned transient buffer -> HBM staging/target KV
```

这条路径必须满足：pinned buffer 的生命周期小于一次 prefetch/drain；所有调度、命中统计和 attention 消费仍以 HBM 中的 block/slot 为准。若机器支持 GDS/cuFile，则优先走 **SSD/NVMe -> HBM**，跳过 CPU bounce。

正确路径：

```
HCA prefill-hit overlap:
  上一层 attention 后 / MoE A2A dispatch 前后
  -> 因 HCA read set 确定，直接提交目标 HCA 层的 SSD/NVMe -> HBM 读
  上一层 expert compute / A2A combine
  -> NVMe DMA 与 MoE 通信/专家计算重叠
  目标 HCA attention 前
  -> drain/sync, 保证 HCA compressed rows 已在 HBM

CSA speculative overlap:
  上一层 attention 后 residual proxy
  -> proxy Lightning Indexer 预测 CSA top-K
  -> SSD/NVMe -> HBM staging 预读 predicted blocks
  目标层 true Lightning Indexer 后
  -> CPU 只比较 true IDs 与 predicted/resident IDs
  -> fallback miss blocks 继续 SSD/NVMe -> HBM
  最终 attention
  -> 始终消费 true LI 转换后的 true slots
```

当前代码中 `lmcache/v1/hca_prefetch_manager.py` 已经从逐 row Python bytes 诊断路径改成 `PinMemoryAllocator` 分配的 transient pinned slab：SSD `pread` 读入 slab，`drain_for_layer()` 在目标 HCA attention 前把 rows 写入 vLLM HCA KV cache，随后释放/复用 pinned buffer。这仍然不是最终性能路径，因为它不是 GDS/cuFile 直达 HBM，还依赖 first-hit seed flat store；但语义上已经满足“CPU pinned 只能是临时 I/O buffer，不能作为 cache”。

当前范围

当前做的是基于 DeepSeek V4 Pro 结构的 HCA/CSA NVMe KV 预取：

```
HCA: deterministic prefetch + budgeted HBM residency
CSA: residual proxy speculative prefetch + true Lightning Indexer miss correction
```

本文只保留和这条路线直接相关的内容。以下内容不属于当前实现主线： 如果某段内容不能回答 HCA deterministic prefetch、CSA residual proxy prefetch、 true miss correction、LMCache/vLLM 挂接或实验复现实操，就不要放进本文。

不可违反的语义

1. 官方 true Lightning Indexer 永远是正确性来源。Speculative prefetch 只能提前搬数据， 不能把 predicted top-K 当成最终 attention top-K。
2. DSv4 的 CSA proxy 输入不是 V2 风格的 `hidden_states + residual`。当前选择的是 attention 后、FFN 前已经可用的 HC-post/residual state。
3. proxy 只决定提前读哪些 block；目标 CSA 层仍然运行官方 true LI，并用 true IDs 做 miss correction。
4. 真实 sparse attention 必须消费 true LI 转换出的 global KV slots。
5. HCA 和 CSA 必须分开调度。HCA 是确定性读，CSA 是 learned-indexer 预测读。
6. chunked prefill 的最终语义必须等价于不切 chunk 的 prefill，所有 token/block ID 必须是 全局 logical ID。
7. 默认安全路径是 prefill 后 KV/IndexerCache 继续留在 HBM。post-prefill eviction 是实验功能， 必须由环境变量显式开启。
8. DSv4 normal LMCache 路径需要完整注册 HCA/CSA 相关 KV tensors。不要把 `kv_caches` 过滤成 61 个主层 cache；正确做法是 V3 GPU connector 识别异构 group。

DSv4 结构事实

DeepSeek V4 Pro 的关键结构：

项目	值
transformer layers	61
CSA layers	30 个，通常是 2, 4, ..., 60
HCA layers	31 个
CSA compress ratio	4
HCA compress ratio	128
CSA index top-K	1024
CSA 机制	stride=4，每个 compressed entry 覆盖约 8-token overlap window，true LI 稀疏选 top-K
HCA 机制	stride=128，dense attention 读全局 compressed entries
SWA 机制	约 128-token 最近窗口，不压缩，保留最近精确状态
KV heads	1, MQA
head dim	512

HCA 和 CSA 的 I/O 模式完全不同：

层类型	读什么	调度方式	HBM 策略
HCA	S / 128 个确定性 compressed entries	提前确定性读	按预算保留或 sliding window
CSA	true LI 选出的 sparse compressed entries	residual proxy 预测读 + true miss 补读	每层保留 hot entries
SWA	最近窗口精确状态	不做远端 sparse prefetch	必须做 eviction/window 策略

单请求 prefix 长度为 S tokens 时，只算 HCA/CSA compressed attention KV 的规模近似为：

```
article simplified entry size = 512 B
CSA = 30 * (S / 4) * 512 ~= 3840 * S bytes
HCA = 31 * (S / 128) * 512 ~= 124 * S bytes
total ~= 3.9 KB/token

vLLM current fp8_ds_mla entry size = 584 B
CSA = 30 * (S / 4) * 584 ~= 4380 * S bytes
HCA = 31 * (S / 128) * 584 ~= 141 * S bytes
total ~= 4.4 KB/token
```

若把 CSA indexer cache 也算入预取状态，当前实现里的 indexer entry 约 132 B：

```
CSA indexer cache = 30 * (S / 4) * 132 ~= 990 * S bytes
attention KV + indexer cache ~= 5.25 KB/token
```

文章里的 Together 口径还给出一个更高层的系统结论：

```
unoptimized, with full SWA storage: about 3.8 KB/token
after SWA eviction strategy: single-node B200 capacity 1.2M -> 3.7M tokens
```

这说明 V4 的省内存优势不是自动成立的。CSA/HCA 本身很小，SWA 是主要瓶颈；必须做 SWA window/eviction，不能把 SWA 按完整长上下文保存。

vLLM 官方 DSv4 实现语义

本地 `F:\vllm_dev` 对应的 DSv4 vLLM 实现是省 HBM 的，但它省的是 native vLLM runtime 里的 KV cache 形态，不是 LMCache 外部复用路径。

官方实现的核心不是把所有 prefix KV 放到 NVMe，而是在 vLLM 内部把 DSv4 的不同状态注册成不同 `KVCacheSpec`：

```
SWA:
DeepseekV4SWACache -> SlidingWindowMLASpec
只需要最近 sliding_window; block manager 可以按滑窗限制 admission。

HCA / CSA attention KV:
DeepseekV4MLAAttention -> MLAAAttentionSpec(compress_ratio=128 or 4)
real_page_size_bytes 按 storage_block_size = block_size / compress_ratio 计算。

CSA indexer cache:
DeepseekV4IndexerCache -> MLAAAttentionSpec(compress_ratio=4, head_dim ~= 132B)
为 Lightning Indexer 保存 compressed indexer entries。

compressor state:
CompressorStateCache -> SlidingWindowMLASpec
只需要继续压缩所需的短窗口状态，不应按完整 prefix 回填到 GPU。
```

对应代码入口：

```
F:\vllm_dev\vllm\models\deepseek_v4\nvidia\ops\attention.py
F:\vllm_dev\vllm\models\deepseek_v4\compressor.py
F:\vllm_dev\vllm\v1\kv_cache_interface.py
F:\vllm_dev\vllm\models\deepseek_v4\nvidia\flashmla.py
F:\vllm_dev\vllm\v1\attention\backends\mla\sparse_swa.py
```

decode 时，FlashMLA sparse attention 消费两类数据：

```
SWA recent window:
swa_cache + decode_swa_indices

compressed global/detail KV:
HCA: deterministic C128A global indices
CSA: true Lightning Indexer top-K -> global KV slots
```

因此，vLLM 官方实现解决的是“GPU 上应该分配和访问哪些 DSv4 KV”。它不是 LMCache/NVMe offload 实现，也没有让 LMCache `retrieve()` 自动理解 SWA 只需尾窗、compressor state 只需边界状态、CSA attention KV 只需 hot entries。

这也是当前“runtime 压缩了，但 LMCache load 仍然大”的根因：压缩语义停在 vLLM `KVCacheSpec` 和 attention backend 内部，generic LMCache bridge 只看到已经注册出来的 tensor groups，并按 logical token chunk 把每个 group 都回填到 GPU。

按当前 7 个 group 粗算每 logical token、每 rank 的回填体量：

SWA cache	$\sim 61 * 584 \text{ B}$	$= 34.8 \text{ KiB}$
HCA attention KV	$\sim 31 * 584 / 128 \text{ B}$	$= 0.14 \text{ KiB}$
HCA compressor state	$\sim 31 * 1024 * 4 \text{ B}$	$= 124 \text{ KiB}$
CSA indexer cache	$\sim 30 * 132 / 4 \text{ B}$	$= 0.97 \text{ KiB}$
CSA indexer compressor state	$\sim 30 * 512 * 4 \text{ B}$	$= 60 \text{ KiB}$
CSA attention KV	$\sim 30 * 584 / 4 \text{ B}$	$= 4.28 \text{ KiB}$
CSA attention compressor state	$\sim 30 * 2048 * 4 \text{ B}$	$= 240 \text{ KiB}$
total	$\sim 464 \text{ KiB/token/rank}$	

真正应该长期加载/驻留在 GPU 的 compressed attention KV 只有 HCA attention KV、CSA attention KV，以及为 true LI 服务的 CSA indexer cache，量级约 **5.4 KiB/token/rank**。大头来自 full-prefix SWA 和三个 float32 compressor state group。native vLLM 通过 [SlidingWindowMLASpec/block manager](#) 把这些状态限制成窗口；LMCache 当前 full-hit retrieve 没有执行这个窗口裁剪，所以才会重新变大。

实现路线判断

当前 generic LMCache 路径按 token chunk 保存 vLLM 注册的全部 KV-like tensors。对 DSv4 来说，这包含 243 个 tensors：SWA cache、HCA/CSA attention KV、CSA indexer cache，以及 多个 float32 compressor state。这个路径适合作为 correctness baseline 和短上下文复用验证，但不可能达到 DSv4 optimized serving 的显存/容量目标。

这里的一阶约束不是 SSD 里存了多少，而是 full-hit retrieve 时有多少数据被 materialize 到 GPU/HBM。SSD 持久化可以保留冗余信息；真正不能接受的是每次命中后把 7 个 group 的完整 prefix chunk 全部 scatter 回 vLLM KV cache。DSv4 optimized mode 必须把“SSD 持久化体量”和“GPU 驻留/本轮加载体量”解耦：

SSD persistent payload:

可以保存较完整的 DSv4 状态，服务 correctness、恢复和离线分析。

GPU resident/load payload:

只能加载当前 prefill/decode 必需的 role-specific subset。

generic LMCache 的 all-groups all-chunks scatter 不能作为 optimized path。

最终路线不是重写整个 LMCache，而是在 LMCache 基础上实现 DSv4-aware KV 管理层：

保留 LMCache：

- token hash / lookup
- SSD/CPU backend
- by_gpu disk sharding
- async I/O、metrics、配置系统

重写/专门化：

- vLLM connector 对 DSv4 7 个 KV groups 的语义识别
- retrieve 命中后不能直接把全部 group 全量 scatter 到 GPU
- SWA 在 GPU 只保留最近窗口；SSD 是否保存完整 SWA 是次要问题
- HCA compressed attention KV 作为确定性全局小 KV 管理，按窗口/预算加载
- CSA compressed attention KV 可完整放 SSD，HBM 只放 true/proxy hot entries
- CSA indexer cache 单独管理，用于 Lightning Indexer scoring，可按可承受预算驻留
- compressor state 只把继续 prefill/decode 所需的边界状态放回 GPU，不回填完整 context

必须由环境变量显式启用 DSv4 optimized mode，不能影响官方 LMCache correctness path：

```
LMCACHE_DSV4_OPTIMIZED_KV=1
```

启用后，容量目标按 optimized payload 估算；关闭时，仍按当前 generic LMCache payload 估算。不要把两套口径混用。

因此实现判断以 GPU load/residency 为准：

可以接受：

SSD 上保存完整 chunk 或冗余 metadata。

不可以接受：

LMCache full-hit 时把 SWA、HCA/CSA compressor state、全量 CSA attention KV 都重新加载进 GPU KV cache。

必须实现：

DSv4 group role -> role-specific loader -> staging/resident set -> attention 使用 true slots。

当前已落地的第一步

文件：F:\LMCache\lmcache\v1\gpu_connector\gpu_connectors.py

VLLMPagedMemGPUConnectorV3.to_gpu() 已经加了 LMCACHE_DSV4_OPTIMIZED_KV=1 控制的 DSv4 role-aware H2D policy。这个版本不改变 LMCache SSD 格式、不改变 hit 计数、不改变 store 路径，只改变 full-hit retrieve 后哪些 group 被回填到 GPU。

当前策略：

```
full transfer:
- HCA compressed attention KV
- CSA compressed attention KV
- CSA indexer cache

tail-only transfer:
- SWA cache
- HCA/CSA/indexer compressor state
```

tail-only 窗口由 LMCACHE_DSV4_OPTIMIZED_TAIL_TOKENS 控制，默认一个 LMCache chunk。它的目的不是最终最优 staging，而是先阻断 full-prefix SWA 和 float32 compressor state 被 generic LMCache retrieve 全量打回 GPU。

后续仍要实现的最终形态：

```
CSA compressed attention KV:
  不应长期全量回填到 vLLM 原 KV cache;
  应改成 SSD -> HBM hot/staging slots, 并让 true LI global slots 指到 staging/BAT。

HCA compressed attention KV:
  可按 deterministic row range 和 resident budget 加载，而不是盲目完整回填。

SWA / compressor state:
  继续保持 window/boundary-only。
```

2026-05-30: HMA block-table correctness 修复

这次修复明确了 DSv4 + LMCache 不能只依赖 tensor shape 推导语义。vLLM HMA 里每个 KVCacheGroup 都有自己的 block table 和 logical block size；LMCache group 必须继承这些 语义，否则 full-hit retrieve 后会把数据写回错误的 GPU KV blocks。

错误路径：

```
cache_config.block_size == 4
-> LMCache 误以为 logical block size 是 4
-> request meta capacity 过小, token_ids 被错误截断
-> full-hit retrieve 只加载部分 tokens 或写错 blocks

shape-only grouping
-> SWA cache 和 CSA attention 这类同 shape、不同 HMA group 的 tensors 被合并
-> 同一个 LMCache group 用错 vLLM block table
-> full-hit 后 attention 读到错误 KV，输出异常
```

修复路径：

```
vLLM service factory
-> 从 vLLM HMA group0 读取 engine logical block size, 当前为 256
-> 写入 layout_hints["inference_engine_logical_block_size"]

vLLM adapter
-> 捕获每个 vLLM kv_cache layer 属于哪个 HMA group
-> 捕获每个 layer 自己的 vLLM logical block size
-> 写入 layout_hints["vllm_kv_cache_group_ids"]
-> 写入 layout_hints["vllm_kv_cache_layer_block_sizes"]

KVLayerGroupsManager
-> LayerGroupIdentity = (vLLM group id, kv_size, heads, head_size, block_size, dtype)
-> 同 shape 但不同 HMA group 的 tensors 分开建 LMCache group
-> 每个 LMCache group 用自己的 vLLM block size 推导 compress_ratio 和 physical_chunk_size

VLLMPagedMemGPUConnectorV3
-> 每个 LMCache group 查回对应 vLLM HMA group 的 block_ids
-> 每个 group 使用自己的 logical block size 做 start/end -> block range
-> DSv4 role-aware H2D policy 在正确分组后执行
```

对应文件：

文件	关键点
F:\LMCache\lmcache\integration\vllm\vllm_service_factory.py	_engine_logical_block_size() 不再使用 DSv4 全局 cache_config.block_size=4
F:\LMCache\lmcache\integration\vllm\vllm_v1_adapter.py	_capture_vllm_hma_layout() 传递 vLLM group id 和 per-layer block size
F:\LMCache\lmcache\v1\kv_layer_groups.py	group identity 加入 vLLM HMA group id，避免同 shape 异语义合并
F:\LMCache\lmcache\v1\gpu_connector\gpu_connectors.py	_hma_block_ids_for_group() 按 group 选择 block table; _dsv4_group_role() 识别 split SWA groups

修复后的 DSv4 group 语义示例：

```
HCA attention KV      -> full transfer, compress_ratio=128
CSA indexer cache     -> full transfer, compress_ratio=4
CSA attention KV      -> full transfer, compress_ratio=4
SWA cache             -> tail-only transfer
compressor state groups -> tail-only transfer
```

验证口径：

```
16K prompt, 64 output, same prompt twice:
  second run full-hit output readable, no corrupted text

16.6K prompt, 512 output, same prompt twice:
  second run full-hit output readable, completion around 507 tokens

logs:
LMCache hit tokens == need to load
Retrieved N out of N required tokens
LMCACHE_DSv4_OPTIMIZED_KV active with HCA/CSA full and SWA/compressor tail-only groups
```

accuracy 当前不额外加日志，按输出文本 sanity 判断：full-hit 后应生成正常语言并能持续 完成 512 token 级别输出。论文级 proxy accuracy 仍使用前面 Experiment 2/4 的专门脚本和 record_attention_topk_slots() 口径。

剩余差距：

- 1. 这一步修的是 correctness 和 H2D 回填语义，不是最终 NVMe BAT/staging-slot 系统。

2. SSD persistent payload 仍可能比最终论文系统大；当前只保证 full-hit 后不把不必要的 full-prefix SWA/compressor state 全量打回 GPU。
3. 日志仍有 direct I/O alignment warning，说明多盘带宽没有被完全干净利用；后续应处理 O_DIRECT 文件大小/offset 对齐或改写块布局。

基础 LMCache 复用数据

历史 CPU/SSD 配置用于说明“复用 KV 比重算快很多”，不是 CSA/HCA prefetch 的最终性能结论。

CPU 配置：

```
chunk_size: 256
local_cpu: true
max_local_cpu_size: 40.0
local_disk: ""
use_layerwise: true
```

SSD 配置：

```
chunk_size: 256
local_cpu: false
max_local_cpu_size: 40.0
local_disk: "/lmcache/nvme0/,...,/lmcache/nvme9/"
local_disk_path_sharding: "by_gpu"
max_local_disk_size: 4096.0
use_layerwise: true
layer_group_size: 1
```

注意：上面的 `/lmcache/nvme*` 是历史实验口径，不能直接作为“8 块物理盘”证据。2026-05-30 复查发现当前容器里的 `/lmcache/nvme*` 可以只是 root 盘上的目录。新的 8 盘实验必须使用真实挂载点：
`/mnt/nvme0,/mnt/nvme2,/mnt/nvme3,/mnt/nvme4,/mnt/nvme5,/mnt/nvme6,/mnt/nvme8,/mnt/nvme9。`

历史加载时间：

length	Recompute	SSD_prev	CPU_curr	vsCPU	vsSSD
16K	0.341	0.129	0.056	6.1x	2.6x
32K	0.756	0.223	0.092	8.2x	3.4x
48K	1.209	0.308	0.126	9.6x	3.9x
64K	1.675	0.390	0.225	7.4x	4.3x
80K	2.540	0.495	0.218	11.7x	5.1x
96K	3.387	0.610	0.259	13.1x	5.6x
112K	4.158	0.696	0.303	13.7x	6.0x
128K	5.093	0.794	0.421	12.1x	6.4x
144K	6.032	0.888	0.437	13.8x	6.8x
160K	7.332	0.997	0.491	14.9x	7.4x
224K	13.442	1.424	0.699	19.2x	9.4x
256K	16.465	1.629	0.805	20.5x	10.1x
320K	25.684	2.056	1.040	24.7x	12.5x

这张表只回答“KV 复用是否值得做”。当前工作进一步回答“DSv4 的 HCA/CSA KV 怎样在 prefill hit 复用阶段从 SSD 提前加载回 HBM，并与层执行重叠”。decode 阶段不是 HCA overlap 的目标：同一请求进入 decode 时，prefix HCA KV 已经在 HBM，HCA 不需要再做 deterministic prefetch。

HCA 流水线

HCA 的读集合由上下文长度确定：

```
block_ids = get_compress_topk_idxseq(seq_len, compress_ratio=128)
```

它不需要 learned Lightning Indexer，也不需要 residual proxy。目标流水线发生在 LMCACHE prefill full-hit / partial-hit 复用阶段：

```
第 N 层即将执行前
-> 提前发第 N+k 个 HCA 层的 deterministic read

第 N 层 attention / FFN / MoE / A2A 执行中
-> 第 N+k 个 HCA 层的 NVMe read 与计算/通信重叠

目标 HCA layer wait_for_layer_load 前
-> HCA KV 已在 HBM
```

HCA 数据量小，但随 context 增长。实现上需要两件事：

1. 用环境变量打开 HCA prefetch，不影响官方路径。
2. 维护 HCA resident set，按预算保留最近将被用到的 compressed entries。

HCA 和 CSA 的关键区别是：HCA 的预取错了，只需要解决何时读、读多少、驻留多少。

当前 HCA pinned-transient prefill-hit 实现边界

当前代码实现的是 prefill-hit 阶段的 HCA deterministic defer 原型：

```
第一次 LMCACHE full-hit retrieve
-> V3 GPU connector 保守执行普通 hca_attention_kv H2D 回填
-> _maybe_seed_hca_reuse_prefetch()
-> HCAPrefetchManager.submit_seed_after_reuse()
-> 每个 rank 从已加载到 HBM 的 HCA kv_cache seed rank-local flat HCA store

后续 LMCACHE full-hit retrieve
-> V3 GPU connector 检测 flat HCA store ready
-> 跳过普通 hca_attention_kv group H2D 回填
-> _maybe_seed_hca_reuse_prefetch() 记录当前请求 compressed row -> physical slot mapping

上一层 FFN 前窗口
-> DeepseekV4DecoderLayer._fire_hca_prefetch(positions)
-> HCAPrefetchManager.fire_async_for_layer(next_hca, positions)
-> rows = deterministic_range((position + 1) // 128)
-> 过滤 HBM resident/pending 后异步 SSD read 到 transient pinned slab

目标 HCA attention 前
-> DeepseekV4DecoderLayer._drain_hca_prefetch()
-> HCAPrefetchManager.drain_for_layer(current_hca)
-> 把 pinned slab 中的 rows 写回目标 HCA layer 的普通 vLLM kv_cache slots
-> 写入 HBM 后才标记 resident_hbm, pinned buffer 不参与 hit/resident

目标 HCA attention
-> 仍使用 vLLM 官方 C128A metadata / global slots
```

这个原型已经能验证“后续 full-hit 不再由 LMCACHE 常规 H2D 回填 HCA group，而是交给 HCA manager 在层执行窗口中补回 HBM”。它仍不是最终性能路径，原因是：

1. flat HCA store 目前由第一次 hit 后从 HBM seed 得到；最终应直接使用 LMCACHE SSD payload 或独立 HCA BAT/offset 文件，不应依赖先完整回填一次。
2. CPU pinned slab 只是 fallback/transient bounce；最终优先接 GDS/cuFile 或 GPU-visible staging slot。
3. 目前用 Python hook 和线程池验证状态机，尚未实现 gio_uring doorbell、GPU DMA gather、TMA/attention 三段硬件流水。

默认不要开启 HCA decode hook。decode 开始时同一请求的 prefix HCA KV 已在 HBM，不需要 HCA prefetch；decode hook 只保留为显式诊断开关。

CSA 流水线

CSA 必须先有 query 才能跑 Lightning Indexer。目标层真实 query 到目标层 attention 才完全可用，所以我们在上一层 attention 后用近似 state 构造 proxy query。

正确窗口：

上一层 attention 完成

- > DSv4 MHC fused post/pre 得到 FFN 前 residual_f / HC-post state
- > 用目标 CSA 层自己的 HC_pre + attn_norm + indexer projection 构造 approximate Q
- > proxy Lightning Indexer 预测 top-1024
- > 在上一层 FFN / expert compute / A2A combine 窗口发 NVMe read
- > 目标 CSA 层运行官方 true Lightning Indexer
- > true IDs - resident/predicted IDs = fallback misses
- > 补读 misses
- > attention 消费 true IDs 对应的 global KV slots

metadata correction 的语义：

```
true_ids = official Lightning Indexer output
spec_ids = proxy predicted output
resident = already in HBM
misses   = true_ids - (spec_ids union resident)

fallback_read(misses)
attention(indices=true_ids_as_global_slots)
prev_topk = true_ids
```

CPU 侧只做 metadata 操作：比较两个 1024 元素集合、生成 miss list、准备 fallback request。实际数据搬运仍然走 NVMe -> HBM 和 HBM 内 gather。

实际调用链

```
LMCacheConnectorV1Impl.register_kv_caches()
-> _attach_indexer_prefetch()
-> _attach_hca_prefetch()
-> indexer_op.ssd_manager = manager
-> indexer_op.csa_layer_id = layer_id
-> DeepseekV4DecoderLayer.attach_indexer_prefetch(manager, next_csa)
-> DeepseekV4DecoderLayer.attach_hca_prefetch(manager, current_hca, next_hca)

上一层 decoder block:
DeepseekV4DecoderLayer._forward_cuda()
-> attention
-> _drain_hca_prefetch() before current HCA attention, if this is an HCA layer
-> MHC fused post-pre for FFN
-> _fire_indexer_prefetch(residual, positions)
-> IndexerSSDManager.fire_async_for_layer(next_csa, residual, positions)
-> _fire_hca_prefetch(positions)
-> HCAPrefetchManager.fire_async_for_layer(next_hca, positions)
-> self.ffn(...)

目标 CSA layer:
SparseAttnIndexer.forward_cuda()
-> torch.ops.vllm.sparse_attn_indexer(...)
-> IndexerSSDManager.insert_decode_token(...)
-> IndexerSSDManager.correct_true_topk(layer_id, true_topk)

真实 sparse MLA attention:
sparse MLA backend
-> triton_convert_req_index_to_global_index(logical true IDs)
-> IndexerSSDManager.record_attention_topk_slots(logical_ids, global_slots)
-> flash_mla_with_kvcache(..., indices=global_slots)
```

这里的 attention backend 只是当前 true slots 的观测点，不改变当前工作目标。

改动文件与职责

文件	关键函数	作用
F:\LMCache\lmcache\integration\vllm\vllm_v1_adapter.py	_attach_indexer_prefetch()	在 LMCache/vLLM 注册 KV cache 后挂载 CSA prefetch manager
F:\LMCache\lmcache\integration\vllm\vllm_v1_adapter.py	_attach_hca_prefetch()	发现 compress_ratio=128 的 HCA layers，并挂载 deterministic prefetch manager
F:\LMCache\lmcache\v1\indexer_ssd_manager.py	fire_async_for_layer()	用上一层 residual 预测目标 CSA top-K，并发异步 SSD read
F:\LMCache\lmcache\v1\indexer_ssd_manager.py	correct_true_topk()	true LI 后 drain 预测读、补 true miss、更新下一轮 true top-K
F:\LMCache\lmcache\v1\indexer_ssd_manager.py	record_attention_topk_slots()	在真实 attention 消费点记录 logical true IDs、global KV slots 和 proxy recall
F:\LMCache\lmcache\v1\hca_prefetch_manager.py	fire_async_for_layer() / drain_for_layer()	按 positions 生成 HCA deterministic rows，异步读并在目标 attention 前写回 vLLM HCA KV cache
F:\LMCache\lmcache\v1\cache_engine.py	_prepare_gpu_connector_layout()	在 memory object 分配前让 V3 connector 发现 DSv4 heterogeneous KV groups
F:\LMCache\lmcache\v1\gpu_connector\gpu_connectors.py	V3 group transfer	按 DSv4 heterogeneous group 搬运 HCA/CSA KV
F:\vllm_dev\vllm\models\deepseek_v4\nvidia\model.py	_fire_hca_prefetch() / _drain_hca_prefetch()	在 NVIDIA DSv4 上一层 FFN/MoE 前提交下一 HCA 的 deterministic read，并在目标 HCA attention 前 drain
F:\vllm_dev\vllm\models\deepseek_v4\amd\model.py	_fire_hca_prefetch() / _drain_hca_prefetch()	AMD 路径同样在 FFN/MoE 前 fire、目标 HCA attention 前 drain
F:\vllm_patch\vllm\model_executor\models\deepseek_v4.py	_fire_hca_prefetch() / _drain_hca_prefetch()	当前容器旧 layout 的实际服务文件；本地 patch 后需要拷回 site-packages/远端 patch 区
F:\vllm_dev\vllm\model_executor\layers\sparse_attn_indexer.py	forward_cuda()	保留官方 true LI，之后插入 decode token

文件	关键函数	作用
		并做 true miss correction
F:\vllm_dev\vllm\v1\attention\backends\mla\flashmla_sparse.py	_record_attention_topk_slots()	把 true logical top-K 和 global KV slots 传回 manager

当前实现边界

已完成或已有原型：

- 1. normal vLLM + LMCacheConnectorV1 + SSD-only 配置已跑通过 DSv4 基础复用。
- 2. V3 GPU connector 能按 DSv4 heterogeneous group 注册和搬运 KV。
- 3. adapter 可由 LMCACHE_INDEXER_ENABLE_PREFETCH=1 挂接 CSA prefetch manager。
- 4. NVIDIA DSv4 hook 放在 attention 后、FFN 前，并传入 residual/positions。
- 5. manager 用目标 CSA 层自己的 HC/attn_norm/indexer 计算 proxy top-K。
- 6. true LI 后做 miss correction，最终 attention 仍消费 true slots。
- 7. LMCache full-hit prefill retrieve 后可用 LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1 seed CSA prototype pool，方便验证 decode 预取链路。
- 8. LMCACHE_HCA_ENABLE_PREFETCH=1 的正确语义是开启 prefill-hit 阶段的 HCA deterministic overlap：按 S/128 全量 HCA compressed rows 提前加载回 HBM。legacy decode fire/drain 原型不能作为性能路径；decode 开始后 prefix HCA KV 已经在 HBM，不需要 HCA prefetch。HCA read set 只由 prefix 长度和 compress_ratio=128 决定，完全复用时不依赖 MoE expert 路由。因此当前实现允许在上一层 attention 完成后、FFN/MoE 开始前提交 下一 HCA 层的 deterministic read，并在目标 HCA attention 前 drain。prefire_first_hca() 仍默认拒绝执行，因为第一层 HCA 在当前 request slot mapping 尚未稳定前不能安全写回。
- 9. vLLM HMA (Hybrid Memory Allocator) 适配：LMCacheConnectorV1 继承 SupportsHMA 并实现 request_finished_all_groups。修复了 RequestTracker.from_new_request 中 block_size 参数错传 lmcache_config.local_cpu (bool) 的 bug，以及下游 token_ids > num_blocks * block_size 时缺少截断保护的 bug。128K 上下文 HMA 已在 容器 dsv4-ep8tp8-128k-hma 端到端验证（17K / 60K / ~117K tokens 均通过）。256K 上下文（容器 dsv4-256k）：当前镜像不含上述修复（均在本地 feature/dsv4-compat 分支），通过容器 entrypoint 内两段 runtime patch 注入（见 Experiment 7 和 runbook）。patch 后 ≤10K tokens 通过；~52K tokens 触发 VLLMPagedMemGPUConnectorV3 contiguous slot offsets 错误，根因是 _build_req_meta() 取 block_ids[0]（HMA 第一个 KV group，并非 full-block-size group）；需要 _select_primary_block_ids(block_size=256) 才能 彻底修复（本地代码已有，当前镜像未包含）。

不能夸大：

- 1. 当前 Python prototype 补的是 CSA IndexerCache records，用于验证状态机和命中率。
- 2. 真正 sparse MLA KV block 的 BAT、SSD offset、HBM staging slot 路径仍未完整落地。
- 3. gio_uring / GPU doorbell / TMA 的完整生产 I/O 管线仍未完成。
- 4. HCA deterministic overlap 的正确落点是 prefill-hit 复用加载路径，不是 decode fire/drain。当前 legacy decode hook 只能说明挂接点存在，不能说明性能收益。
- 5. 真正的 HCA prefill-hit overlap 仍需要接入 LMCache layerwise / V3 connector 调度，让后续 HCA 层的 deterministic rows 在当前层计算时加载。
- 6. chunked prefill 等价状态机仍需要专门处理。
- 7. AMD/ROCm residual proxy 传参仍不是展示路径。

实验结果

Experiment 1: Delta-selection 稳定性

问题：N 个 decode step 中，CSA Indexer 选中的 compressed blocks 有多少比例与上一步保持一致？

Model	Prompt len	Steps	Mean overlap
V4-Flash	32768	300	90.5%
V4-Pro	32768	500	91.0%

V4-Pro 逐层结果：

- Layer 10 最低，约 71.9%。
- 大多数层在 88% 到 97%。
- 所有 CSA 层均值约 91.0%。

工程含义：每一步不应全量重读 1024 个 CSA blocks。HBM hot set 可以沿 decode 逐步演化，每层每步大约只变化 92/1024 个 blocks。

Experiment 2: proxy 准确率

问题：能否用上一层 attention/FFN 窗口期间可得的 proxy 预测目标 CSA 层 Indexer 选择？

Proxy	计算方式	V4-Flash	V4-Pro
hc_post(shared)	HC_post(shared_experts(x_ffn), residual_f, post_f, comb_f)	92.5%	92.9%
attn+residual	attention 后、FFN 前的 residual/HC-post state	87.8%	83.8%

当前选择 `attn+residual`，原因不是它最精确，而是它最早可用。它在 attention 完成后即可触发，能覆盖完整 FFN 窗口，约 3300 us。
`hc_post(shared)` 精度更高，但窗口更短。

Experiment 3: decode steps 中的选择漂移

Step T	adj_overlap	long_overlap
1	0.906	0.906
5	0.902	0.866
10	0.907	0.852
25	0.907	0.824
50	0.903	0.787
100	0.909	0.723

结论：

- 相邻步 overlap 稳定在约 0.906，说明 delta-selection 每一步都有效。
- 相对 step0 的 long overlap 会缓慢下降，100 steps 后仍有约 72.3% 保留。
- resident set 是逐步漂移，不是突然失效。

Experiment 3.5: CSA block 的跨层复用分析（C3）

问题：同一个 decode step 内，CSA layer L 与 L+1 选中的 compressed block 位置是否高度重合？如果高度重合，才可以把“跨层共享 hot residency”作为强假设推进。

实验配置：

- Date: 2026-06-02
- Host: gpu002 (172.16.8.32)
- Container image: lmcache/vllm-openai:indexer-ssd-hca-prefetch-decodegate-20260528_0630
- Model: V4-Pro, TP=8, 使用 /mnt/nvme0/models/DeepSeek-V4-Pro/model{0..7}-mp8.safetensors
- Prompt len: 32768 tokens
- Decode steps: 50
- CSA layers: 30
- index top-K: 1024
- Artifacts:
 - tmp_gpu002_logs/csa_cross_layer_summary_20260602.json
 - tmp_gpu002_logs/csa_cross_layer_blocks_20260602.jsonl
 - tmp_gpu002_logs/csa_cross_layer_20260602.log

Metric	Observed	Random independent baseline	Notes
same-layer adjacent-step overlap	0.906 mean	N/A	与 Experiment 3 的 0.906 对齐
adjacent-layer overlap L -> L+1	0.166 mean / 0.146 p50 / 0.279 p90	0.125	只略高于随机
adjacent-layer Jaccard	0.093 mean	0.067	只略高于随机

Metric	Observed	Random independent baseline	Notes
all-layer naive selected blocks / step	30720	30720	30 layers * 1024
all-layer unique numeric block ids / step	7864.7 mean	8042.8	比随机少约 1.8%
numeric-id reuse factor	3.906x	3.820x	主要来自有限 block-id 空间，不是强跨层相关性

相邻层中 overlap 最高和最低的代表：

Layer pair	overlap mean	Jaccard mean
2 -> 4	0.435	0.278
44 -> 46	0.310	0.184
4 -> 6	0.281	0.163
50 -> 52	0.274	0.158
6 -> 8	0.071	0.037
32 -> 34	0.078	0.041
30 -> 32	0.083	0.043

结论：

- C3 的“相邻层高 overlap”假设 **不成立**。相邻层均值 0.166 只比随机基线 0.125 略高，不能支撑“L 的 hot set 可直接复用给 L+1”这类强优化。
- 30 层合并后的 unique numeric block-id 只有约 7865，看起来有 3.9x 去重；但随机基线 已经有 3.82x。因此这个数字主要由 8192 个 block 位置的有限全集导致，不能单独解释为 跨层选择高度一致。
- 更稳妥的策略是：继续把 **decode step 方向的 delta-selection** 作为主收益来源；跨层方向 只作为全局 HBM manager 的调度/统计信号，而不是假设层间 KV payload 可以共享。注意： 相同 numeric block id 在不同层对应不同层的 KV 数据，不能被解释为一份 HBM payload 的物理共享。

Experiment 4: partial KV cache hit 下的 proxy 准确率

配置：

- Model: V4-Pro
- Prompt len: 32768
- Decode steps: 50
- index top-K: 1024

K / L	K tokens	proxy_acc
100%	32768	0.839
75%	24576	0.849
50%	16384	0.872
25%	8192	0.919
10%	3276	1.000

结论：

- proxy 精度随 prefill 缩短而上升。
- partial KV cache hit 不会损害 proxy 精度。
- partial hit 场景 fallback 成本更低。
- 困难层主要是 layer 8、54、56，需要更大的 fallback budget 或特殊策略。

Timing profile

Operation	Time
dist/all-to-all 窗口	150-200 us
shared experts forward	约 80 us
HC_pre + attn_norm	123.8 us
HC_post	约 30 us
完整 proxy pipeline	约 230 us
NVMe read 保守估计	约 300 us
完整 FFN 窗口	约 3300 us

这个 timing 支持当前设计：即使用 124-230 us 做 proxy，也能换到足够长的 NVMe overlap 窗口。

环境变量

主开关：

```
LMCACHE_INDEXER_ENABLE_PREFETCH=1
LMCACHE_INDEXER_SSD_DIR=/mnt/nvme0/csa_ssd_pool/indexer
LMCACHE_INDEXER_POOL_SIZE=4096
LMCACHE_INDEXER_IO_WORKERS=8
LMCACHE_INDEXER_MAX_SEQ_LEN=131072
```

residual proxy：

```
LMCACHE_INDEXER_ENABLE_RESIDUAL_PROXY=1
```

LMCache full-hit 后 seed CSA prototype pool：

```
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_INDEXER_REUSE_PREFETCH_MAX_TAIL_TOKENS=4096
```

LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0 是性能默认值。它允许 full-hit prefill reuse 后 seed CSA prototype pool，但不在 decode 的每个 token、每个 CSA 层运行 Python residual proxy、decode-token insert、drain 和 true-miss fallback。把它设为 **1** 只用于 accuracy/状态机诊断；最终高性能路径必须把这部分换成 BAT/gio_uring/staging slot，而不是 Python 循环补读。

诊断：

```
LMCACHE_INDEXER_PROFILE_ACCURACY=1
LMCACHE_INDEXER_TIMING=1
```

默认关闭：

```
LMCACHE_INDEXER_ENABLE_PREFILL_EVICTION=0
LMCACHE_INDEXER_PREFETCH_PREFILL_ROWS=0
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
```

HCA 独立开关，不能复用 CSA residual proxy 的含义：

```
LMCACHE_HCA_ENABLE_PREFETCH=1
LMCACHE_HCA_SSD_DIR=/mnt/nvme0/csa_ssd_pool/hca
LMCACHE_HCA_PREFETCH_WINDOW_TOKENS=0
```

```
LMCACHE_HCA_PREFETCH_MAX_BLOCKS=0
LMCACHE_HCA_RESIDENT_BUDGET_BLOCKS=0
LMCACHE_HCA_IO_WORKERS=8
LMCACHE_HCA_MAX_SEQ_LEN=131072
LMCACHE_HCA_TIMING=1
LMCACHE_HCA_BLOCKING_DRAIN=0
```

PREFETCH_WINDOW_TOKENS=0 和 PREFETCH_MAX_BLOCKS=0 表示按当前位置需要的全部 HCA compressed rows 做 deterministic prefetch。非 0 时用于限制尾部窗口，便于在长上下文 上控制 prototype I/O 量。

Experiment 5: HCA legacy decode hook smoke（2026-05-29，非性能结论）

容器：dsv4-hca-overlap（image indexer-ssd-hca-prefetch-decodegate-20260528_0630，TP8，max-model-len 32768，LMCACHE_HCA_ENABLE_PREFETCH=1，LMCACHE_HCA_TIMING=1）。

测试序列（prompt ~5120 tokens）：

Phase	描述	结果
Phase 1	初次 prefill（存入 SSD）	1.1 s
Phase 2	第二次 prefill（LMCache full-hit → seed HCA）	0.8 s
Phase 3	decode 64 tokens（legacy fire/drain hook 启动）	7.7 s

关键日志观察：

```
# 启动
HCAPrefetchManager: enabled deterministic HCA prefetch on 61 decoder layers
and attached 31 HCA caches, store=.../hca/rank_0
(all 8 TP ranks)

# Phase 2 full-hit → seed
HCAPrefetchManager: reuse prefetch seeded 31 HCA layers
lmcache_tokens=5120 compressed_tokens~=40
(all 8 TP ranks)

# Phase 3 decode: legacy hook 日志
HCAPrefetchTiming: event=fire layer=33 total_ms=0.5 rows=40 missing=0
HCAPrefetchTiming: event=drain layer=33 total_ms=0.002 pending=0 written=0
```

结论：

- 这只验证了 legacy decode hook 的 attach/seed/fire/drain 状态机，不是 HCA prefill-hit overlap 性能验证。
- rows=40：5120 / 128 = 40 compressed rows，符合 HCA compress_ratio=128。
- missing=0：seed 阶段已把所有 40 rows 标记为 resident，fire 阶段没有发 NVMe 读，因此不能证明 I/O overlap。
- LMCACHE_HCA_BLOCKING_DRAIN=0 默认 non-blocking drain，row 写回延迟 取决于 NVMe 读完成时间，不阻塞 attention。
- 当前结论：HCA prefill-hit overlap 还没有被这组实验验证；后续实验必须看 LMCache full-hit/partial-hit 阶段的 per-layer load 与模型层执行重叠。

Experiment 6: vLLM HMA 128K 上下文验证（2026-05-29）

容器：dsv4-ep8tp8-128k-hma（image 含 SupportSHMA 和 request_finished_all_groups 修复，TP8，max-model-len 131072）。

prompt tokens	耗时	结果
17,500	3.9 s	SUCCESS
60,001	7.3 s	SUCCESS
116,902	12.9 s	SUCCESS

日志中无 num tokens > num blocks * block_size、无 AssertionError、无 Traceback。说明 HMA _select_primary_block_ids 的 block_size 参数 bug 修复有效。

修复点（F:\LMCache\lmcache\integration\vllm\vllm_v1_adapter.py）：

1. `RequestTracker.from_new_request` 新增 `block_size: Optional[int] = None` 参数，不再把 `lmcache_config.local_cpu` (bool) 传给 `_select_primary_block_ids`。
2. `build_connector_meta` 调用点改传 `block_size=self._block_size`。
3. `token_ids > num_blocks * block_size` 时改为截断 + `logger.warning`，而非只 `log error`。

Experiment 7: 256K 上下文容器 token 梯度测试 (2026-05-29)

容器: `dsv4-256k` (image `indexer-ssd-hca-prefetch-decodegate-20260528_0630`, TP8, `max-model-len 262144`, `--no-disable-hybrid-kv-cache-manager`, `use_layerwise: false`, `use_gpu_connector_v3: true`, `YAML chunk_size: 256`)。镜像不含 `SupportsHMA/request_finished_all_groups/token-clamping` 修复 (均在本地 `feature/dsv4-compat` 分支)，通过容器 `entrypoint` 中两段 `inline Python patch` 注入：

1. **patch_connector.py** (修改容器内 `lmcache_connector.py`)：向 `LMCacheConnectorV1` 加 `SupportsHMA` 继承和 `request_finished_all_groups(block_ids: tuple[list[int], ...])` 方法。用途：`use_layerwise: false` 下 `vLLM scheduler` 看到多个 KV group，若 `connector` 不继承 `SupportsHMA` 则触发 `assert len(kv_cache_groups) == 1`；`SupportsHMA` 绕过该断言，通过 `request_finished_all_groups` 把 `primary group (block_ids[0])` 转发给 `_lmcache_engine.request_finished()`。
2. **patch_adapter.py** (修改容器内 `vllm_v1_adapter.py`)：在 `_build_req_meta()` 的 `len(token_ids) > num_blocks * block_size` 分支加截断 `token_ids = token_ids[:max_capacity]` (并同步更新 `tracker.num_saved_tokens`)；防止 `wait_for_save()` 触发 `assert len(slot_mapping) == len(token_ids)`。

token 梯度测试结果 (模型 ID `deepseek-v4-pro`, `served-model-name`):

prompt tokens	结果	说明
~65	OK	
~260	OK	此前因 <code>assert</code> 失败， <code>patch</code> 后通过
~2,600	OK	
~10,400	OK	
~52,000	FAIL	<code>ValueError: VLLMPagedMemGPUConnectorV3 block transfer requires contiguous slot offsets within each inference-engine block</code>

52K 失败根因: `_build_req_meta()` 取 `block_ids[0]` (`vLLM HMA` 分配的第一个 KV group，并非 full 256-token block 的 group)，而 `_slot_mapping_to_block_ids()` 在大 长度下 `block` 分配出现不连续时无法满足 `slot[i*logical_block_size+k] % logical_block_size == k`。彻底修复需本地 `feature/dsv4-compat` 分支的 `_select_primary_block_ids(block_size=256)` 被构建进 镜像，当前镜像无此函数。

Experiment 8: 256K LMCache 基准 + prefetch ON vs OFF 对比 (2026-05-29)

容器: `dsv4-256k` (image `indexer-ssd-hca-prefetch-decodegate-20260528_0630`, TP8, `max-model-len 262144`, `--no-disable-hybrid-kv-cache-manager`，历史配置使用 10 路 LMCache disk `/lmcache/nvme{0..9}`)/。该路径后来被证明不是可靠 8 盘配置，见 Experiment 10)，`use_layerwise: false`, `use_gpu_connector_v3: true`, `chunk_size: 256`)。两段 `runtime patch` (`patch_connector.py` + `patch_adapter.py`，同 Experiment 7) 在启动时注入。

两种条件 (其余 `env` 完全相同)：

- **prefetch ON**: `LMCACHE_INDEXER_ENABLE_PREFETCH=1`, `LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1`
- **prefetch OFF**: `LMCACHE_INDEXER_ENABLE_PREFETCH=0`, `LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=0`

其他固定 `env`：

- `LMCACHE_DSV4_OPTIMIZED_KV=1`：只存 HCA/CSA attention KV + IndexerCache，每 rank payload 从 ~464 KiB/token (全 7 group) 降到 ~21.8 KiB/token，255K tokens 共 ~5.5 GB/rank。
- `LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0`：decode 期 CSA 预取关闭。
- HCA 预取关闭 (`LMCACHE_HCA_ENABLE_PREFETCH=0`, `LMCACHE_HCA_SSD_DIR=` 为空)。

prompt: `"gradient test token Q. " * 51000` (5 tok/rep × 51000 = 255,002 tokens，已通过 API 确认 `prompt_tokens=255006`)。

A. TTFT (gen=1)

条件	测项	prompt tokens	elapsed	说明
prefetch ON	cold (首次 store)	~255K	34.320 s	无缓存, 纯 prefill 计算
prefetch ON	full-hit-1	~255K	3.709 s	LMCache SSD 全命中, load + scatter + reuse seed
prefetch OFF	cold (首次 store)	~255K	40.996 s	无缓存, 纯 prefill 计算 (无 reuse seed)
prefetch OFF	full-hit-1	~255K	19.494 s	LMCache SSD 全命中, 纯 SSD load, 无 prefetch
prefetch OFF	full-hit-2	~255K	19.516 s	稳定重复, 方差 <0.1%

注: cold 数据两者相差约 6.7 s, 推测是 prefetch ON 的 reuse seed 阶段有 warm-up 效果或测量时 GPU 温度/调度状态不同, 不影响 full-hit 对比结论。

B. LMCache payload / SSD profile (prefetch ON 条件, 两次相同)

- Retrieved 254976 out of 254976 required tokens (LMCache 全命中)
- payload: **5.5057 GB/rank** (DSV4_OPTIMIZED_KV=1, 否则 ~118 GB/rank)
- SSD 读取速度: 1.65 s (热盘, ~3.3 GB/s) ~ 3.8 s (冷盘, ~1.4 GB/s)
- reuse prefetch seeded 30 CSA layers, compressed_tokens=63744 (= 255K / 4)

C. CSA evict_after_prefill 逐层计时 (IndexerSSDTiming, prefetch ON)

通过 /tmp/lmcache_indexer_timing 文件 flag (无需重启容器) 开启, 日志格式: event=evict_after_prefill layer=<N> seq_len=<M> load_ids=4096 total_ms=... read_ms=... write_ms=... load_ms=...

采样 (seq_len=63744, load_ids=4096):

指标	典型范围
total_ms (含 read + write + load)	40 ~ 84 ms
read_ms (主导项, SSD → CPU buffer)	33 ~ 49 ms
write_ms (IndexerCache 写回)	1 ~ 3 ms
load_ms (CPU buffer → GPU scatter)	3 ~ 40 ms

30 CSA 层串行 evict_after_prefill 合计约 1.2 ~ 2.5 s (实际被 LMCache SSD retrieve 覆盖, 不在关键路径上)。

结论

- DSV4_OPTIMIZED_KV=1 是 256K 可行的核心前提: payload 从 ~118 GB → ~5.5 GB/rank, SSD read 时间从不可用降至 1.6-3.8 s。
- prefetch ON full-hit TTFT: **3.709 s** (vs cold 34.3 s, LMCache 加速 9.3×)。
- prefetch OFF full-hit TTFT: **19.5 s** (vs prefetch ON, 慢 5.3×)。
- prefetch ON 加速来源: REUSE_PREFETCH 在 cold prefill 后把 CSA 数据 evict_to_SSD 的同时 预热 seed pool; full-hit 时大部分 SSD load 与 prefill 重叠, 只剩 scatter overhead ~3.7 s。
- prefetch OFF full-hit 的 19.5 s ≈ 5.5 GB/rank × 10 ranks 的 SSD 顺序读取时间 (冷盘), 印证 payload 分析: 无预取就是纯 SSD I/O 延迟。
- vLLM HMA 兼容性 fix (PATCH5/6/8/9) 已验证: prefetch OFF 全 3 轮不再 crash (原 run 2 因 _slot_mapping_to_block_ids contiguous check 失败而 crash, 修复后稳定通过)。

Experiment 9: 修复后 58K ON/OFF 重新对比 (2026-05-30)

Claude 之前的 ON/OFF 对比不再作为结论使用。本实验在 HMA block-table correctness 修复后, 用 dsv4-256k 容器重新测:

共同条件:
LMCACHE_DSV4_OPTIMIZED_KV=1
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_HCA_ENABLE_PREFETCH=0
max_tokens=1
served model: deepseek-v4-pro

OFF:
LMCACHE_INDEXER_ENABLE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=0

```
ON:
LMCACHE_INDEXER_ENABLE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1
```

条件	prompt tokens	cold elapsed	full-hit elapsed	retrieve
OFF	58,801	7.922 s	3.262 s	58,624 tokens, 25.951 GB/rank, 2.388-2.599 s/rank
ON	57,401	7.399 s	3.646 s	57,344 tokens, 25.384 GB/rank, 2.210-3.227 s/rank

ON 日志出现：

```
IndexerSSDManager: reuse prefetch seeded 30 CSA layers ...
```

但因为本实验显式保持 `LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0`，ON 的收益不会在 `max_tokens=1` 的 TTFT 中体现。当前 ON 只是做 full-hit 后 CSA prototype pool seed，因此 58K 下比 OFF 慢约 0.38 s 是合理的。

可信结论：

- 1. HMA 修复后的 full-hit 基础复用稳定，输出 sanity 正常。
- 2. `LMCACHE_DSV4_OPTIMIZED_KV=1` 下 58K retrieve 达到约 80 GB/s 级别整机有效吞吐（LMCache 日志口径、8 ranks 并行）。
- 3. CSA reuse seed 当前是为后续 decode 预取准备状态，不应声称它在 gen=1 TTFT 中已经加速。
- 4. 要证明 CSA prefetch 真正加速，需要打开 decode 阶段 prefetch 或完成 BAT/gio_uring/ staging-slot 路径后，测长 decode 的 first-token/steady-token latency。

Experiment 10: 真实 8 盘 full-hit 与磁盘级带宽验证（2026-05-30）

目的：确认 LMCache SSD-only full-hit 是否真的使用 8 块物理 NVMe，而不是只写到 8 个目录。

修正：容器和 YAML 改为真实数据盘路径：

```
/mnt/nvme0/lmcache_dsv4_cache/
/mnt/nvme2/lmcache_dsv4_cache/
/mnt/nvme3/lmcache_dsv4_cache/
/mnt/nvme4/lmcache_dsv4_cache/
/mnt/nvme5/lmcache_dsv4_cache/
/mnt/nvme6/lmcache_dsv4_cache/
/mnt/nvme8/lmcache_dsv4_cache/
/mnt/nvme9/lmcache_dsv4_cache/
```

启动日志确认 `local_disk_path_sharding: by_gpu` 后 8 个 ranks 分别选择 8 块盘：

```
cuda:0 -> /mnt/nvme0/lmcache_dsv4_cache/
cuda:1 -> /mnt/nvme2/lmcache_dsv4_cache/
cuda:2 -> /mnt/nvme3/lmcache_dsv4_cache/
cuda:3 -> /mnt/nvme4/lmcache_dsv4_cache/
cuda:4 -> /mnt/nvme5/lmcache_dsv4_cache/
cuda:5 -> /mnt/nvme6/lmcache_dsv4_cache/
cuda:6 -> /mnt/nvme8/lmcache_dsv4_cache/
cuda:7 -> /mnt/nvme9/lmcache_dsv4_cache/
```

请求配置：

```
prompt_tokens: 238,245
max_tokens: 8
LMCACHE_DSV4_OPTIMIZED_KV=1
LMCACHE_HCA_ENABLE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
```

结果：

测项	结果
cold	40.372 s
hot full-hit	3.228 s
drop-cache full-hit	3.878 s
LMCache hit tokens	238,080 / 238,245
retrieve payload	4.5823 GB/rank
hot retrieve	1.24-1.29 s/rank, 3.56-3.69 GB/s/rank
drop-cache retrieve	1.90-1.95 s/rank, 2.35-2.41 GB/s/rank
iostat peak aggregate read	27.66 GB/s
iostat per disk	3.43-3.50 GB/s, all 8 disks active

验证方法：

```
sync
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
iostat -dxm 1 30 > /tmp/dsv4_8disk_dropcache_iostat.log &
# 立即发送同一个 full-hit prompt
```

结论：

- 1. 真实 8 盘已经同时参与读取，之前 `/lmcache/nvme*` 口径不能作为 8 盘证据。
- 2. 单盘当前只有约 3.5 GB/s 峰值，说明瓶颈已经从“没分盘”转移到 `local_disk` Python/file-read、每 rank 单线程顺序读、CPU bounce 与 H2D pipeline。
- 3. 若要继续提高带宽，下一步应做 per-rank 内部并发读、更大连续块，或切到 GDS/cuFile / BAT + gio_uring + HBM staging slot。

Experiment 11: prefill full-hit CSA/HCA 开关对比（2026-05-30）

目的：只测 prefill full-hit/retrieve 阶段，不测 decode prefetch。所有请求使用 `max_tokens=1`，并显式关闭 decode 期 CSA/HCA hook：

```
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_HCA_ENABLE_DECODE_HOOK=0
```

运行位置：

```
host: gpu002, 172.16.8.32
container: dsv4-256k
endpoint: http://127.0.0.1:8000
results: /tmp/dsv4_prefill_compare/summary.jsonl
```

三组开关：

组	CSA reuse prefetch	HCA overlap	其他关键开关
A	off	off	LMCACHE_INDEXER_ENABLE_PREFETCH=0, LMCACHE_HCA_ENABLE_PREFETCH=0
B	on	off	LMCACHE_INDEXER_ENABLE_PREFETCH=1, LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1, LMCACHE_HCA_ENABLE_PREFETCH=0
C	on	on	B + LMCACHE_HCA_ENABLE_PREFETCH=1, LMCACHE_DSV4_DEFER_HCA_TO_MOE=1, LMCACHE_HCA_ENABLE_PINNED_BOUNCE=1

结果：

组	prompt tokens	cold	full-hit 1	full-hit 2	LMCache hit/load	retrieve payload	retrieve cost/rank	retrieve throughput/rank	输出
A: CSA off, HCA off	253,845	38.460s	25.439s	25.501s	253,696	112.303 GB	20.28-24.68s	4.55-5.54 GB/s	Answer, Pref
B: CSA on, HCA off	248,644	38.142s	22.121s	22.301s	248,576	110.037 GB	19.55-21.21s	5.19-5.63 GB/s	Answer
C: CSA on, HCA on	243,443	37.095s	24.810s	24.268s	243,200	107.657 GB	19.44-22.21s	4.85-5.54 GB/s	Answer

8 盘 iostat -dxm 峰值读带宽：

组	hit1 aggregate	hit2 aggregate	单盘峰值范围
A	46.81 GB/s	46.60 GB/s	5.61-6.11 GB/s
B	46.30 GB/s	46.28 GB/s	5.57-6.08 GB/s
C	46.26 GB/s	46.46 GB/s	5.62-5.98 GB/s

日志结论：

1. A/B/C 都是真 full-hit：Inference Engine computed tokens: 0，LMCache hit tokens 与 need to load 等于对齐后的 prompt token 数。
2. B 出现 IndexerSSDManager: reuse prefetch seeded 30 CSA layers，说明 CSA prefill reuse seed 路径已经触发。B 的 full-hit TTFT 比 A 低约 3.2 s，但本次三组 prompt 文本含 group label，token 数不完全相同，只能作为工程趋势，不能作为论文最终数字。
3. C 出现 HCAPrefetchManager: reuse prefetch seeded 31 HCA layers、fire、drain 和 mode=pinned_transient，说明 HCA overlap 代码跑到了。
4. C 同时出现大量 LMCACHE_DSV4_DEFER_HCA_TO_MOE=1 but HCA flat store is not ready; keeping normal HCA H2D transfer 日志，本轮 hit1/hit2 分别仍有正常 HCA H2D fallback，因此 HCA overlap 没有形成可见收益。当前不能把 C 解释为 HCA overlap 加速成功，只能说 hook/state machine 已接入，数据准备时机还不对。
5. 本次对比实验的 LMCache Retrieved ... size: 是 107-112 GB/rank。注意这不是 LMCACHE_DSV4_OPTIMIZED_KV 被关掉了：日志已经确认 LMCACHE_DSV4_OPTIMIZED_KV active 和 KV layer groups 正确加载。当前实现的优化点在 full-hit retrieve 之后的 DSv4 role-aware H2D/GPU 回填，而不是 SSD 持久化格式本身；LMCache 仍会先从 SSD 取完整 MemoryObj 到 CPU/pinned transient buffer，再按 HCA/CSA/SWA 角色选择写回 GPU。因此 Retrieved size 仍然反映完整 LMCache 磁盘对象体量，不等于最终 GPU 回填体量。
6. Experiment 10 中记录的 4.582 GB/rank 与本轮 107-112 GB/rank 口径冲突，不能继续 当作同一类 Retrieved size 直接比较。后续要单独复核 4.582 GB/rank 是哪一层口径：SSD read、H2D copied bytes，还是经过 role-aware selection 后的有效 GPU payload。

后续修复优先级：

1. HCA flat store 必须在第一轮 full-hit retrieve 前可用；否则 defer HCA H2D 会频繁 fallback。
2. C 组需要把 HCAPrefetchManager 的 seed 与 LMCache retrieve metadata 对齐，确保同一请求内 HCA compressed rows 已在 flat store 中可查。
3. 用完全相同的 prompt 重跑 A/B/C。当前 runner 把 group label 写进 prompt，导致 token 数 分别为 253,845、248,644、243,443，影响精确对比。
4. 复核 Experiment 10 的 4.58 GB/rank 口径，避免把 SSD retrieve bytes 与 role-aware H2D/GPU 回填 bytes 混在一起。

2026-05-30 HCA flat-store seed 修复

修复点：

1. HCAPrefetchManager 新增 seed_range_from_lmcache_group()，直接从 LMCache retrieve 出来的 HCA group tensor 写入 rank-local flat HCA store，按 chunk 的 logical token offset 转成 compressed row offset。
2. VLLMPagedMemGPUConnectorV3.to_gpu() 遇到 hca_attention_kv group 且 LMCACHE_DSV4_DEFER_HCA_TO_MOE=1 时，先把本 chunk 写入 flat store，再跳过普通 HCA H2D 回填。这样不再依赖“第一次 full-hit 先完整 H2D，然后从 HBM 反拷 seed”的慢路径。
3. adapter 的 _maybe_seed_hca_reuse_prefetch() 如果发现 flat store 已经由 retrieve chunk 准备好，只记录当前 request 的 compressed row -> physical slot mapping，不再用旧的 kv_cache.detach().cpu() 覆盖 flat store。

这个修复只解决 HCA fallback：目标日志应从 HCA flat store is not ready; keeping normal HCA H2D transfer 变成 seeded HCA flat store directly from LMCache retrieve and deferred normal HCA H2D。

它不会单独把 256K full-hit 从 22 s 降到论文目标，因为当前最大慢点是 SSD backend 仍可能读取旧的完整 LMCache MemoryObj。验证这次修复和 optimized store 体量时必须清空旧 LMCache cache path 或换新 cache 目录，否则会继续命中旧 full-object，Retrieved size 仍会显示 100GB/rank 级别。

2026-05-30 runtimefix smoke：完整补丁进入容器后

确认 dsv4-256k 之前慢的直接原因之一是启动脚本没有覆盖完整 LMCache runtime patch tree： 它只覆盖 adapter/gpu_connector/HCA/KVLayerGroups，漏了 cache_engine.py 等 optimized store/retrieve 文件。已修改远端 /tmp/startup_256k.sh，让 /patches/v1/、 /patches/integration/ 和 /patches/utils.py 全量覆盖 site-packages。

为避免旧 full-object cache 干扰，复测使用新 8 盘目录：

```
/mnt/nvme{0,2,3,4,5,6,8,9}/lmcache_dsv4_cache_runtimefix/
```

容器内确认：

```
has_seed_range True
has_old_ready_msg False
has_store_shapes True
```

60K/256K smoke，max_tokens=1，第二轮前 drop cache：

prompt tokens	cold	drop-cache full-hit	LMCache hit/load	Retrieved size/rank	retrieve cost/rank
60,001	7.583 s	1.201 s	59,904	1.2036 GB	0.50-0.53 s
252,001	33.168 s	3.052 s	251,904	4.7653 GB	2.07-2.12 s

旧 64K 曲线的 Retrieved size/rank 是 27.878 GB，full-hit TTFT 是 6.201 s。完整补丁和 新 cache path 后，同量级 prompt 已降到 1.2036 GB/rank、1.201 s。旧 256K 曲线的 Retrieved size/rank 是 111.283 GB，full-hit TTFT 是 22.342 s；runtimefix 后是 4.7653 GB/rank 和 3.052 s。这说明必须区分：

- 1. 新 optimized store 写出的对象体量已经显著变小。
- 2. 旧 cache path 里的 full-object 仍会让 Retrieved size 回到 100GB/rank 级别。
- 3. 后续所有性能对比必须记录 cache path 和补丁加载校验，否则数字不可比。

Experiment 12: CSA on 多长度 full-hit TTFT 曲线（2026-05-30）

目的：只看 CSA prefill reuse seed 开启时，full-hit TTFT 随 prompt 长度的变化。固定：

```
LMCACHE_INDEXER_ENABLE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_HCA_ENABLE_PREFETCH=0
LMCACHE_HCA_ENABLE_DECODE_HOOK=0
max_tokens=1
drop_caches before hit
```

运行位置：

```
host: gpu002, 172.16.8.32
container: dsv4-256k
results: /tmp/dsv4_csa_on_curve/summary.json1
docker log: /tmp/dsv4_csa_on_curve/docker_20260530_140102.log
```

结果：

case	actual prompt tokens	cold TTFT	full-hit TTFT	LMCache hit/load tokens	LMCache Retrieved size/rank	retrieve cost/rank	retrieve throughput/rank	output
16K	15,547	4.995 s	2.433 s	15,360	6.799 GB	1.23-1.33 s	5.11-5.55 GB/s	Answer
32K	31,527	3.568 s	3.618 s	31,488	13.939 GB	2.49-2.66 s	5.24-5.61 GB/s	Answer
64K	63,017	7.373 s	6.201 s	62,976	27.878 GB	4.99-5.28 s	5.28-5.59 GB/s	Answer
128K	125,997	15.769 s	11.455 s	125,952	55.755 GB	10.00-10.58 s	5.27-5.57 GB/s	Answer
192K	188,977	31.418 s	16.896 s	188,928	83.632 GB	15.11-15.86 s	5.27-5.53 GB/s	Answer
256K	251,487	34.748 s	22.342 s	251,392	111.283 GB	20.17-21.47 s	5.18-5.52 GB/s	Answer

日志证据：

1. 6 个长度均为 full-hit：Inference Engine computed tokens: 0，LMCache hit tokens 等于对齐后的 need to load。

2. 6 个长度均出现 IndexerSSDManager: reuse prefetch seeded 30 CSA layers，说明 CSA prefetch reuse seed 路径每轮都触发。

3. Retrieved size 随 token 数线性增长，约 0.443 MiB/token/rank，对应当前 LMCache SSD MemoryObj 完整对象读取口径。这个数字仍不是 GPU role-aware 回填后的有效 payload。

4. 16K/32K 的 cold TTFT 波动不完全单调，可能受 warmup、已有 page cache、调度噪声影响；长度 >=64K 后趋势稳定。

5. full-hit TTFT 基本由 SSD retrieve 时间主导；retrieve throughput/rank 在 5.18-5.61 GB/s 区间内稳定。

Experiment 13: 同 prompt CSA on/off full-hit 对比（2026-05-30）

Experiment 12 只跑了 CSA on，不是严格对比。本实验使用同一批 prompt 文本（run_id=20260530_140102）补跑 CSA off。注意：由于 LMCache 使用 builtin hash，跨容器重启后旧 cache key 不可复用；OFF 第一轮 LMCache hit tokens: 0，只是重新 store，不是 full-hit。下面表格使用 OFF 同容器第二轮结果，即真正 full-hit：

```
OFF valid logs:
LMCache hit tokens == need to load
CSA_SEED 0

ON valid logs:
LMCache hit tokens == need to load
CSA_SEED 6
```

case	tokens	CSA on full-hit	CSA off full-hit	on - off	结论
16K	15,547	2.433 s	1.726 s	+0.707 s	on 慢，seed 开销主导
32K	31,527	3.618 s	3.060 s	+0.558 s	on 慢，seed 开销主导
64K	63,017	6.201 s	5.620 s	+0.581 s	on 慢，seed 开销仍未被摊薄
128K	125,997	11.455 s	11.665 s	-0.210 s	基本持平
192K	188,977	16.896 s	17.779 s	-0.883 s	on 快 5.0%
256K	251,487	22.342 s	24.101 s	-1.759 s	on 快 7.3%

OFF retrieve 日志：

case	hit/load tokens	Retrieved size/rank	retrieve cost/rank	throughput/rank
16K	15,360	6.799 GB	1.37-1.45 s	4.69-4.98 GB/s
32K	31,488	13.939 GB	2.69-2.89 s	4.83-5.18 GB/s
64K	62,976	27.878 GB	5.03-5.40 s	5.17-5.54 GB/s
128K	125,952	55.755 GB	10.77-11.29 s	4.94-5.18 GB/s

case	hit/load tokens	Retrieved size/rank	retrieve cost/rank	throughput/rank
192K	188,928	83.632 GB	16.72-17.21 s	4.86-5.00 GB/s
256K	251,392	111.283 GB	22.55-23.30 s	4.78-4.93 GB/s

结论：

1. 当前 CSA prefetch reuse seed 对 TTFT 的收益很小：短上下文因为 seed 自身开销反而变慢，长上下文 192K/256K 才出现约 5-7% 端到端收益。
2. 这不是最终设计预期的收益形态。当前 full-hit TTFT 明显太长，主因是 SSD 层仍读取完整 LMCACHE MemoryObj：256K 时每 rank Retrieved size 为 111.283 GB，retrieve 本身 约 20-23 s。
3. 真正要把 TTFT 压下来，需要把“少读”推进到 SSD I/O 层，即只读 HCA/CSA/SWA-tail 所需 ranges 或 block，而不是先完整读到 CPU/pinned buffer 后再 role-aware H2D 少写。
4. builtin hash 会导致跨容器重启后的相同 prompt cache key 不稳定。做 hit-only 对比时，必须在同一容器内先 store 再测第二轮，或改成稳定 hash 算法。

2026-05-31 runtimefix 后 HCA-on 复测

目的：在完整 runtime patch 已进入 dsv4-256k、并使用 /mnt/nvme{0,2,3,4,5,6,8,9}/lmcache_dsv4_cache_runtimefix/ 新 cache path 后，重新验证 HCA pinned-transient overlap 是否能在 252K full-hit 上继续降低 TTFT。

固定条件：

```
prompt: "Runtime fix 256K validation prompt. " * 31500
prompt_tokens: 252,001
max_tokens: 1
drop_caches before hit
LMCACHE_INDEXER_ENABLE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_HCA_ENABLE_PREFETCH=1
LMCACHE_HCA_ENABLE_DECODE_HOOK=0
LMCACHE_HCA_ENABLE_PINNED_BOUNCE=1
LMCACHE_HCA_PINNED_BUFFER_MB=64
LMCACHE_HCA_BLOCKING_DRAIN=0
```

启动配置修复记录：

1. 一开始把 LMCACHE_HCA_PINNED_BUFFER_MB=2048 和 LMCACHE_HCA_BLOCKING_DRAIN=1 写进启动脚本，容器在 API ready 前卡住，主 PID 后续变成 zombie，docker exec 也失败。改回 64 MiB 和 non-blocking drain 后 容器可正常 ready。
2. 之后发现 YAML 里的 extra_config.dsv4_defer_hca_to_moe 仍为 false，虽然环境变量 LMCACHE_DSV4_DEFER_HCA_TO_MOE=1 已设置。这会让 HCA manager 额外 fire/drain，但普通 HCA H2D 不被跳过，数字不可作为有效 HCA-on 结论。
3. 有效 HCA-on 对比必须同时满足： LMCACHE_DSV4_DEFER_HCA_TO_MOE=1 且 YAML extra_config.dsv4_defer_hca_to_moe: true。

结果：

配置	run	LMCache hit/load	full-hit elapsed	retrieve size/rank	retrieve cost/rank	关键日志
HCA off runtimefix 基线	hit	251,904	3.052 s	4.7653 GB	2.07-2.12 s	HCA disabled
HCA on, 但 YAML defer=false	hit2	251,904	3.740 s	4.7653 GB	2.09-2.12 s	reuse prefetch seeded, 但无 direct defer
HCA on, 但 YAML defer=false	hit3	251,904	3.651 s	4.7653 GB	2.10-2.12 s	HCA 成为额外工作
HCA on, YAML defer=true	hit2	251,904	4.411 s	4.7653 GB	2.82-2.83 s	direct seed + deferred normal HCA H2D
HCA on, YAML defer=true	hit3	251,904	4.280 s	4.7653 GB	2.78-2.80 s	direct seed + fire/drain

有效 defer=true 的日志确认：

```
LMCACHE_DSV4_DEFER_HCA_TO_MOE=1:
  seeded HCA flat store directly from LMCache retrieve and deferred normal HCA H2D
HCAPrefetchManager: reuse prefetch seeded 31 HCA layers ...
HCAPrefetchManager: fire layer=0 rows=1968 missing=1968 mode=pinned_transient
HCAPrefetchManager: drain layer=0 written=1968 mode=pinned_transient
```

拆分观察：

指标	YAML defer=false	YAML defer=true
retrieve mean/rank	2.104 s	2.804 s
HCA fire events	960	384
HCA fire total logged time	2.883 s	1.160 s
HCA drain total logged time	9.000 s	3.458 s
direct flat-store seed	0	8
fallback flat store is not ready	0	0

结论：

- 1. 当前 HCA pinned-transient Python 原型在 252K full-hit 上没有收益；有效 defer=true 后 反而比 HCA-off 慢约 1.2-1.4 s。
- 2. 慢点不是 HCA 没触发，而是触发方式不划算：HCA attention KV 本身很小， 省掉普通 HCA H2D 的收益不足以抵消 retrieve 阶段 direct seed flat store、层间 Python fire/drain、以及 pinned slab 写回目标 HBM 的开销。
- 3. 这进一步说明当前 HCA overlap 不应继续沿 Python manager + flat-store seed 做性能优化。 下一步性能路线应转向真正的 HCA payload layout： 直接从 LMCACHE/HCA SSD range 或 BAT/gio_uring/GDS 读到 GPU-visible staging，避免在 full-hit retrieve 中先建立 Python flat store，再逐层写回普通 vLLM KV cache。
- 4. 在当前 runtimefix 路径下， 优先保留 HCA off 作为性能基线；HCA on 只用于状态机诊断。

2026-06-01 direct request multi-length full-hit check

目的：按最简单端到端请求口径重测当前修正后 full-hit 曲线，并复测 CSA on + HCA on。本节不使用 drop_caches，不删除已经写入的 KV cache； 每个长度先发一次 cold/store 请求，等待 60 s，然后直接对同一 prompt 发 full-hit 请求。不同长度、不同组之间使用不同 prompt 前缀，避免跨长度 prefix/key 混用。

固定条件：

```
host: gpu002, 172.16.8.32
container: dsv4-256k
served model: deepseek-v4-pro
max_tokens: 1
TP: 8
local_disk_path_sharding: by_gpu
local_disk: /mnt/nvme{0,2,3,4,5,6,8,9}/...
extra_config.use_odirect: false
local_cpu: false
use_layerwise: false
```

开关：

```
A: csa_off_hca_off / off_current_direct:
  LMCACHE_INDEXER_ENABLE_PREFETCH=0
  LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=0
  LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
  LMCACHE_HCA_ENABLE_PREFETCH=0
  LMCACHE_HCA_ENABLE_DECODE_HOOK=0

B: csa_on_hca_off / csa_on_hca_off_direct:
  LMCACHE_INDEXER_ENABLE_PREFETCH=1
  LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1
  LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
```



```
LMCACHE_HCA_ENABLE_PREFETCH=0
LMCACHE_HCA_ENABLE_DECODE_HOOK=0

C: csa_on_hca_on / csa_on_hca_on_direct:
LMCACHE_INDEXER_ENABLE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_HCA_ENABLE_PREFETCH=1
LMCACHE_HCA_ENABLE_PINNED_BOUNCE=1
LMCACHE_DSV4_DEFER_HCA_TO_MOE=1
LMCACHE_HCA_ENABLE_DECODE_HOOK=0
```

原始结果文件：

```
/tmp/dsv4_direct_multilen_results.jsonl
```

三组 direct full-hit 对比：

target	A off/off mean	B CSA on/HCA off mean	C CSA on/HCA on mean	A tokens	B tokens	C tokens
32K	0.397 s	0.333 s	0.347 s	29,545	29,544	32,008
64K	0.728 s	0.639 s	0.607 s	68,934	68,936	64,012
128K	1.366 s	1.221 s	1.239 s	137,857	137,857	137,857
192K	1.712 s	1.746 s	1.708 s	177,240	192,010	192,010
248K	2.249 s	2.163 s	2.150 s	248,438	248,439	248,440

A: CSA off + HCA off:

target	prompt tokens	cold/store	full-hit runs	full-hit mean
32K	29,545	3.207 s	0.346 / 0.448 s	0.397 s
64K	68,934	7.767 s	0.609 / 0.848 s	0.728 s
128K	137,857	16.265 s	1.188 / 1.545 s	1.366 s
192K	177,240	21.754 s	1.491 / 1.933 s	1.712 s
248K	248,438	32.459 s	1.979 / 2.519 s	2.249 s

B: CSA on + HCA off:

target	prompt tokens	cold/store	full-hit runs	full-hit mean
32K	29,544	3.224 s	0.313 / 0.378 / 0.308 s	0.333 s
64K	68,936	7.660 s	0.588 / 0.740 / 0.589 s	0.639 s
128K	137,857	16.293 s	1.120 / 1.424 / 1.118 s	1.221 s
192K	192,010	23.886 s	1.588 / 2.045 / 1.605 s	1.746 s
248K	248,439	32.513 s	1.984 / 2.531 / 1.974 s	2.163 s

C: CSA on + HCA on:

target	prompt tokens	cold/store	full-hit runs	full-hit mean
32K	32,008	3.463 s	0.325 / 0.397 / 0.320 s	0.347 s
64K	64,012	7.130 s	0.560 / 0.697 / 0.564 s	0.607 s
128K	137,857	16.243 s	1.134 / 1.447 / 1.136 s	1.239 s
192K	192,010	23.797 s	1.580 / 1.988 / 1.555 s	1.708 s

target	prompt tokens	cold/store	full-hit runs	full-hit mean
248K	248,440	32.489 s	1.966 / 2.504 / 1.980 s	2.150 s

第二组 CSA on + HCA on 复测，使用新的唯一 prompt 前缀：

target	prompt tokens	cold/store	full-hit runs	full-hit mean
32K	32,007	3.426 s	0.320 / 0.387 / 0.316 s	0.341 s
64K	64,012	7.057 s	0.544 / 0.678 / 0.541 s	0.588 s
128K	137,858	16.301 s	1.124 / 1.416 / 1.114 s	1.218 s
192K	206,780	26.646 s	2.175 / 2.658 / 2.231 s	2.355 s
248K	248,440	32.612 s	1.962 / 2.527 / 2.022 s	2.170 s

观察：

1. 在直接第二遍请求口径下，248K full-hit 为 A 2.249 s、B 2.163 s、C 2.150 s，三组很接近，B/C 比 A 略快但幅度不大。
2. 32K/64K/128K 也表现为 B/C 略快；192K 第二组 prompt 实际为 206,780 tokens，不应和第一组 192,010 tokens 做严格逐点比较。
3. B 与 C 在 248K 上只有 0.013 s mean 差异；这组端到端数据不能证明 HCA on 带来独立收益，只能说明当前开关组合没有显著拖慢 direct full-hit。
4. 这组结果只说明当前 direct request full-hit 路径稳定在 2 s 量级；它不证明 HCA overlap 已形成独立大收益，因为请求没有拆分 retrieve/fire/drain 的内部耗时。
5. 后续若要证明 HCA 贡献，需要在同一 prompt/token 数下采集 LMCache retrieve、HCA seed/fire/drain 计时和日志事件，而不是只看端到端 elapsed。

2026-06-01 已有分组件 profile 汇总

本节只记录已经从日志或前序实验中拿到的分组件数字。不要把这些数字解释成完整 HCA/CSA 生产路径性能；当前仍是 Python prototype + pinned-transient 路径。

组件	场景	已有数字	说明
LMCache retrieve	252K runtimefix HCA off	2.07-2.12 s/rank	Retrieved size/rank=4.7653 GB，full-hit elapsed 3.052 s
LMCache retrieve	252K HCA on defer=false	2.09-2.12 s/rank	HCA manager 有额外工作，但普通 HCA H2D 未跳过
LMCache retrieve	252K HCA on defer=true	2.78-2.83 s/rank	direct seed flat store 后跳过普通 HCA H2D，但 retrieve 阶段变慢
HCA fire	252K defer=false	960 events, total logged 2.883 s	旧口径，HCA 成为额外工作
HCA drain	252K defer=false	total logged 9.000 s	旧口径，跨层/跨 rank 事件求和，不能直接加到端到端
HCA fire	252K defer=true	384 events, total logged 1.160 s	有效 direct seed + defer 口径
HCA drain	252K defer=true	total logged 3.458 s	有效 direct seed + defer 口径
HCA direct seed	252K defer=true	8 flat-store seeds	每个 TP rank 一次 direct flat-store seed
CSA evict_after_prefill	256K prefetch ON 旧诊断	40-84 ms/layer	read_ms=33-49, write_ms=1-3, load_ms=3-40
CSA evict_after_prefill	30 CSA layers	约 1.2-2.5 s total	这是实验 post-prefill eviction path，当前 prefill-only 性能测试应关闭

2026-06-03 all-off 248K retrieve 组件 profile

目的：回答 all-off 条件下 full-hit TTFT 的组件拆分。之前 all-off 只有端到端 TTFT 和 LMCache Retrieved ... cost/throughput，没有拆出 LMCache SSD/object load 与 H2D/scatter。

运行位置：

```
host: gpu002
container: dsv4-256k-guard
startup: G all_off_profile
prompt: ALLOFF_REPEAT2_248K_4HIT_20260602_UNIQUE_PREFIX + sentence * 17745
prompt_tokens: 248,462
max_tokens: 1
cache root: /mnt/nvme{0,2,3,4,5,6,8,9}/lmcache_dsv4_codex_D_20260601_153415/
use_odirect: false
LMCACHE_INDEXER_ENABLE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_RESIDUAL_PROXY=0
LMCACHE_HCA_ENABLE_PREFETCH=0
LMCACHE_DSV4_DEFER_HCA_TO_MOE=0
```

Instrumentation：临时把 LMCache `RetrieveRequestStats` 打到日志：`process_tokens_ms`、`to_gpu_ms`、`broadcast_ms`、`other_ms`。这里 `process_tokens_ms` 覆盖 cache key/token 处理、storage manager 读取 MemoryObj、重排 chunk 等 CPU/SSD 对象加载路径；`to_gpu_ms` 覆盖 `gpu_connector.batched_to_gpu()` 的 H2D/scatter。

Artifacts:

```
tmp_gpu002_logs/dsv4_alloff_profile_extra_248k_20260603.jsonl
tmp_gpu002_logs/dsv4_alloff_profile_extra_248k_20260603.docker.log
```

三次 full-hit:

hit	elapsed	LMCache retrieve total/rank	process_tokens/rank	to_gpu/rank	other/rank	size/rank
1	3.566 s	2.618 s	1.669 s	0.947 s	0.002 s	4.7469 GB
2	2.176 s	1.334 s	0.383 s	0.950 s	0.002 s	4.7469 GB
3	2.118 s	1.327 s	0.375 s	0.949 s	0.002 s	4.7469 GB

steady hit2/hit3 均值:

Component	Mean	Fraction of LMCache retrieve
LMCache retrieve total	1.331 s/rank	100%
process_tokens / SSD object load	0.379 s/rank	28.5%
H2D/scatter (<code>batched_to_gpu</code>)	0.949 s/rank	71.4%
broadcast	0.000 s/rank	0%
other	0.002 s/rank	0.2%

结论:

- 1. all-off steady 248K full-hit 的 LMCache retrieve 内部主项是 **H2D/scatter**，约 0.95 s/rank，占 retrieve 约 71%。
- 2. SSD/object load (`process_tokens_ms`) 在 steady hit 中约 0.38 s/rank；hit1 的 1.67 s 是慢样本，和端到端 3.56 s 对齐，说明 variance 仍可能来自 storage/process 侧抖动。
- 3. all-off 端到端 2.1-2.2 s 中，LMCache retrieve steady 约 1.33 s/rank，剩余约 0.8 s 是 vLLM scheduling、metadata、first decode/model execution、HTTP 等端到端 包络。
- 4. 这组 profile 说明：若目标是在 all-off/基础复用路径继续降 TTFT，只优化 SSD read 不够；role-aware H2D/scatter、目标 KV 写入和 staging layout 是更大的稳定瓶颈。

2026-06-03 CSA attach-on / decode-off 248K retrieve 组件 profile

目的：回答 `CSA on` 在当前安全性能口径下到底影响哪一块。这里不开启 `LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH`，也不创建 `/tmp/lmcache_indexer_enable_prefill_eviction`。因此本实验只测 CSA manager attach、residual proxy 配置存在时，LMCache full-hit retrieve 的组件拆分；它不测旧 `evict_after_prefill` 路径，也不把 post-prefill seed 解释成 prefill overlap。

运行位置:

```
host: gpu002
container: dsv4-256k-measure
prompt: CSA_ATTACH_ON_PROFILE_248K_20260603_UNIQUE_PREFIX + sentence * 17745
prompt_tokens: 248,458
max_tokens: 1
cache root: /mnt/nvme{0,2,3,4,5,6,8,9}/lmcache_dsv4_codex_D_20260601_153415/
use_odirect: false
LMCACHE_INDEXER_ENABLE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_RESIDUAL_PROXY=1
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_PREFILL_EVICTION=0
LMCACHE_INDEXER_PREFETCH_PREFILL_ROWS=0
LMCACHE_HCA_ENABLE_PREFETCH=0
LMCACHE_DSV4_DEFER_HCA_TO_MOE=0
```

Artifacts:

```
tmp_gpu002_logs/dsv4_csa_attach_on_retrieve_profile_248k_20260603.jsonl
tmp_gpu002_logs/dsv4_csa_attach_on_profile_20260603.docker.log
```

端到端请求结果:

run	elapsed	prompt tokens
cold/store	35.920 s	248,458
hit1	2.313 s	248,458
hit2	3.211 s	248,458
hit3	2.185 s	248,458

按 8 个 TP ranks 聚合的 LMCache retrieve profile:

hit	retrieve total/rank	process tokens/rank	to_gpu/rank	other/rank	size/rank
1	1.385 s	0.426 s	0.957 s	0.002 s	4.7469 GB
2	2.423 s	1.467 s	0.954 s	0.002 s	4.7469 GB
3	1.383 s	0.427 s	0.953 s	0.002 s	4.7469 GB

日志事件计数:

event	count
IndexerSSDTiming	0
reuse prefetch seeded	0
event=fire_async	0
event=correct_true_topk	0
evict_after_prefill	0
attention_true_topk	0

与 all-off steady hit2/hit3 对比:

Condition	steady retrieve total/rank	process tokens/rank	to_gpu/rank	结论
all-off	1.331 s	0.379 s	0.949 s	基线
CSA attach-on, decode-off	1.384 s	0.427 s	0.955 s	形态相同，差值约 50 ms/rank

判断：

1. 当前安全性能口径下，CSA attach/residual-proxy 配置没有触发真实 CSA I/O：`fire_async`、`correct_true_topk`、`reuse_prefetch_seeded`、`evict_after_prefill` 均为 0。
2. `to_gpu_ms` 仍稳定在约 0.95 s/rank，和 all-off 完全同形；慢样本来自 `process_tokens_ms` 从约 0.43 s/rank 抖到约 1.47 s/rank。
3. 因此这组数据不能支持“CSA 在 prefill hit 里隐藏了一段额外 I/O 开销”。当前能看到的只是基础 LMCACHE retrieve 抖动；真正要测 CSA prefill overlap，需要一个独立 prefill-only profile hook：上一层 attention 后提交 CSA 预测读，到目标层 true topK 处记录 `window_ms` / `wait_ms` / `hidden_ms`，并且继续保持 `LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0`、不启用 post-prefill eviction。

2026-06-03 CSA prefill-overlap wait profile (profile-only)

目的：补上前一节缺失的对应 wait log。该实验在 prefill 阶段成对记录：

1. `prefill_overlap_fire`：在 `fire_async_for_layer()` 中提交 profile-only 异步读。
2. `prefill_overlap_drain`：在 attention/indexer true topK 记录点 drain pending reads，记录 `wait_ms` / `async_span_ms` / `window_ms` / `hidden_ms` / `hidden_ratio`。

运行位置：

```
host: gpu002
container: dsv4-256k-measure
prompt_tokens: 248,461
max_tokens: 1
cache root: /mnt/nvme{0,2,3,4,5,6,8,9}/lmcache_dsv4_codex_D_20260601_153415/
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_PREFILL_EVICTION=0
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=0
LMCACHE_HCA_ENABLE_PREFETCH=0
LMCACHE_INDEXER_PREFILL_OVERLAP_PROFILE=1
LMCACHE_INDEXER_PREFILL_OVERLAP_PROFILE_LIMIT=512
LMCACHE_INDEXER_PREFILL_OVERLAP_READ_LIMIT=256
LMCACHE_INDEXER_PREFILL_OVERLAP_ROWS=1
```

Artifacts：

```
tmp_gpu002_logs/dsv4_prefill_overlap_248k_profile_20260603.jsonl
tmp_gpu002_logs/dsv4_prefill_overlap_248k_profile_20260603.log
tmp_gpu002_logs/dsv4_prefill_overlap_248k_profile_20260603.log.gz
tmp_gpu002_logs/dsv4_prefill_overlap_smoke_44k_20260603.log.gz
```

端到端请求结果：

run	elapsed	prompt tokens
248K cold/store + profile	36.244 s	248,461

日志事件计数：

event	count
event=prefill_overlap_fire	4,096
event=prefill_overlap_drain	2,048

248K `prefill_overlap_drain` 聚合：

metric	mean	p50	max
wait_ms	0.506 ms	0.522 ms	1.492 ms
async_span_ms	6.673 ms	5.292 ms	506.772 ms
window_ms	39.780 ms	25.495 ms	515.850 ms

metric	mean	p50	max
hidden_ms	6.167 ms	4.758 ms	506.415 ms
hidden_ratio	0.905	0.902	0.999

补充指标：

metric	value
pending reads per drain	512
candidate hit ratio against current true topK	0.132

解释：

- 对这次 profile 采样的 candidate reads 来说，真正 drain 到 attention 处还需要等的时间 是 sub-ms：平均 `wait_ms=0.506ms`，p50 `0.522ms`。
- 被 prefill window 覆盖的读时间约为 `async_span_ms - wait_ms`：平均 `hidden_ms=6.167ms`，平均隐藏比例 `hidden_ratio=90.5%`。
- 这不是端到端 TTFT 节省，也不是完整 1024-block production CSA 路径收益。为避免在 prefill residual-proxy kernel 上触发 Triton illegal memory access，本次 profile 使用 上一次已记录 true topK 作为候选读源，只回答“这类 candidate read 放在 prefill window 里能隐藏多少 wait”；`candidate hit ratio=13.2%` 也说明它不能证明 residual proxy 的 选择准确率。
- 实验全程保持 `LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0`，`LMCACHE_INDEXER_ENABLE_PREFILL_EVICTION=0`，没有走 `evict_after_prefill`。

HCA 生效性日志也已经在 gpu002 上重新确认。命令：

```
sudo docker logs dsv4-256k 2>&1 | grep -E \  
'enabled pinned-transient|DEFER_HCA_TO_MOE|reuse prefetch prepared|HCAPrefetchManager:  
fire|HCAPrefetchManager: drain|HCAPrefetchTiming' | tail -200
```

2026-06-01 现场日志显示 HCA drain 确实在跑，例如 248K 级别请求里多个 TP worker 出现：

```
HCAPrefetchManager: drain layer=11 written=1936 mode=pinned_transient  
HCAPrefetchManager: drain layer=13 written=1936 mode=pinned_transient  
...  
HCAPrefetchManager: drain layer=59 written=1936 mode=pinned_transient
```

`written=1936` 对应约 `1936 * 128 = 247,808` logical tokens，和 248K 级别 full-hit 请求匹配。因此当前问题不是 HCA 完全未生效，而是 HCA pinned-transient 路径生效后没有形成端到端收益。

当前缺口：

- 6/1 direct full-hit 多长度实验只有端到端 elapsed 和 HCA drain 事件日志，缺 `HCAPrefetchTiming` 的 seed/fire/drain 细分。
- 若要继续 profile，必须在同一 prompt/token 数下开启 `LMCACHE_HCA_TIMING=1` 或容器内 timing flag，采集 `seed_lmcache`、`fire`、`drain` 的 `total_ms/submit_ms/wait_ms/write_ms`，同时保留 LMCache retrieve size/cost。
- prefill-only 性能口径下必须删除或关闭 `/tmp/lmcache_indexer_enable_prefill_eviction`，否则会把实验性的 `evict_after_prefill` 计入请求路径或污染日志判断。

2026-06-01 HCA timing 打开后的 248K profile

运行位置：

```
host: gpu002  
container: dsv4-256k  
condition: D / CSA off + HCA on  
CACHE_SUFFIX=codex_D_20260601_153415  
prompt: "The quick brown fox jumped over the lazy dog near the river bank. " * 17744  
+ " one two three four five six seven eight nine ten"  
prompt_tokens: 248,427  
max_tokens: 1  
drop_caches: no
```

```
LMCACHE_INDEXER_ENABLE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_HCA_ENABLE_PREFETCH=1
LMCACHE_HCA_ENABLE_PINNED_BOUNCE=1
LMCACHE_DSV4_DEFER_HCA_TO_MOE=1
LMCACHE_HCA_ENABLE_DECODE_HOOK=0
LMCACHE_HCA_TIMING=1
LMCACHE_HCA_TIMING_LIMIT=8192
```

结果文件：

```
/tmp/dsv4_hca_timing_248k_20260601.jsonl
/tmp/dsv4_hca_timing_248k_logs.txt
```

端到端结果：

run	prompt tokens	elapsed	LMCache hit/load	retrieve size/rank	retrieve cost/rank
cold/store	248,427	36.090 s	0	-	-
hit1	248,427	3.257 s	248,320	4.7469 GB	1.410-1.514 s
hit2	248,427	5.023 s	248,320	4.7469 GB	3.369-3.456 s

hit2 变慢主要来自 LMCACHE retrieve 本身从约 1.46 s/rank 变成约 3.42 s/rank， 不是 HCA drain wait 变长。

HCA timing 聚合：

event	count	sum total_ms	mean total_ms	关键字段
seed_lmcache	15,520	2,357.173 ms	0.152 ms	每条写 2 rows, layers=31
fire all	1,440	3,305.321 ms	2.295 ms	包含 no-op fire
fire pending=1	496	630.660 ms	1.271 ms	rows=1940, missing=1940, pending=1
fire no-op	944	2,674.661 ms	2.833 ms	missing=0, pending=0
drain	496	8,994.380 ms	18.134 ms	written=1940, pending=1

HCA drain 分项：

field	count	sum	mean	min	max
select_ms	496	2.127 ms	0.004 ms	0.003 ms	0.011 ms
wait_ms	496	0.575 ms	0.001 ms	0.001 ms	0.006 ms
write_ms	496	8,846.637 ms	17.836 ms	15.882 ms	102.818 ms

HCA fire pending=1 分项：

field	count	sum	mean	min	max
slots_ms	496	517.229 ms	1.043 ms	1.016 ms	1.206 ms
filter_ms	496	63.552 ms	0.128 ms	0.053 ms	5.428 ms
submit_ms	496	38.977 ms	0.079 ms	0.061 ms	0.745 ms

判断：

1. HCA 确实生效：hit 请求里有 8 rank DEFER_HCA_TO_MOE direct seed， 16 条 reuse prefetch prepared 31 HCA layers， 248 条 fire 和 248 条 drain info 日志；timing 中也有 496 条 pending drain (hit1 + hit2)。
2. 这次 HCA 的 SSD read 等待几乎完全被隐藏：drain wait_ms 总和只有 0.575 ms。
3. 当前主要额外成本是把 pinned-transient rows 写回目标 KV cache：drain write_ms 总和 8.846 s、均值 17.8 ms/event。这个是跨 rank、跨层事件求和， 不能直接加到端到端，但它解释了为什么 HCA on 很难形成明显收益。

4. **fire** 有大量 no-op (944/1440) 仍然要花 slot mapping/filter 相关 Python 时间。即使没有 pending I/O, **fire total_ms** 合计仍有 2.675 s。
5. 所以后续优化方向不是继续证明 HCA 有没有触发, 而是减少/消除 Python no-op fire、pinned buffer -> KV cache 写回, 以及从 LMCACHE object 再构造 flat store 的 seed 开销。真正路径仍应转向 HCA SSD range/BAT/GDS/GPU-visible staging。

2026-06-01 long-prompt generation sanity after direct full-hit tests

目的: 性能数字之外, 确认开启 CSA/HCA 后长 prompt 生成内容是否稳定, 避免只看 **elapsed** 而漏掉 KV cache 写回错位、时序污染或状态机竞态。

C 组重新启动并确认 HCA overlapping 确实开启:

```
container: dsv4-256k
CACHE_SUFFIX=codex_C_20260601_103604
LMCACHE_HCA_ENABLE_PREFETCH=1
LMCACHE_HCA_ENABLE_PINNED_BOUNCE=1
LMCACHE_DSV4_DEFER_HCA_TO_MOE=1
LMCACHE_HCA_ENABLE_DECODE_HOOK=0
LMCACHE_INDEXER_ENABLE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1
extra_config.dsv4_defer_hca_to_moe: true
HCAPrefetchManager: enabled pinned-transient HCA state
```

同一个约 5.5K prompt tokens 的英文/中文总结类长 prompt, 在 C 组连续请求两次:

run	elapsed	现象
C_RUN 1	39.264 s	主体结构正常, 但出现异常片段 去打2.5s
C_RUN 2	37.275 s	明显跑偏, 输出 JSON/采样参数片段、 电量不足, 请充电后继续使用 、以及 < begin_of_file_name > 文件块

随后切到 B 组 (CSA on / HCA off) 测试同类长 prompt:

```
LMCACHE_INDEXER_ENABLE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_HCA_ENABLE_PREFETCH=0
IndexerSSDManager: enabled CSA prefetch
```

B 组中文 prompt 连续两次:

run	elapsed	现象
B_RUN 1	38.972 s	主体结构正常, 但出现 去打 2.上一 2.5 秒
B_RUN 2	36.580 s	主体结构正常, 但出现 约 一本秒

B 组英文 prompt 多参数复测:

case	参数	现象
greedy_512	temperature=0, max_tokens=512	基本正常, 但输出前自动插入 instruction + </think>
greedy_256	temperature=0, max_tokens=256	基本正常, 但同样带 </think>
greedy_512_stop_eof	temperature=0, stop 文件结束符	多数正常, 但出现 similarEDIFF , 后面又开始第二段总结
temp001_512	temperature=0.01, max_tokens=512	明显退化为重复指令

观察:

1. 生成异常不是 HCA on 独有; B 组只开 CSA 也会出现异常 token、重复指令或文件块风格输出。
2. C 组第二次长 prompt 明显跑偏, 且该轮已证明 HCA on 环境真实生效, 因此不能只看 2 s 量级 full-hit 延迟就断言语义正确。
3. 异常表现受 **max_tokens**、**temperature**、**stop** 影响, 但 **temperature=0** 仍不能完全消除。

4. 这提示当前 CSA/HCA prefetch-hit seed、HCA defer、KV 写回或服务内部状态之间可能存在 **时序问题或状态污染风险**。下一步必须用 A/B/C 三组同 prompt、同参数、同 seed/stop 做 cold-vs-hit 内容一致性对照，并同时抓取 LMCache hit/load、CSA seed、HCA seed/fire/drain 日志，不能仅依据端到端 elapsed 判断正确性。

验证口径

最小正确性证据：

- 1. DSv4 服务能启动，prefetch 只在环境变量开启时挂接。
- 2. 同 prompt 第二次请求出现 LMCache Retrieved / LMCache hit tokens。
- 3. reuse prefetch seeded 30 CSA layers 出现，说明 full-hit prefetch 后 CSA prototype pool 初始化。
- 4. decode 阶段出现 fire_async、correct_true_topk、record_attention_topk_slots。
- 5. true attention 消费的是 true LI 转换出的 global slots，而不是 predicted IDs。
- 6. HCA 开启时出现 HCAPrefetchManager: enabled、seeded、fire、drain 日志。

性能要拆开看：

- LMCache load time / throughput。
- TTFT / prefetch time。
- first decode token latency。
- steady decode latency。
- proxy compute time。
- async read drain / fallback read time。
- post-prefill pool initialization time。

只看端到端 elapsed 会把 prefetch、LMCache load、decode 生成和 prototype 初始化混在一起。

当前 runtime snapshot

2026-05-28 30K/512 可比性能结果：

同一台 gpu002、同一个 30,000 token prompt、max_tokens=512、TP8、--enforce-eager、--gpu-memory-utilization 0.88、--no-enable-prefix-caching。LMCache 为 SSD-only、use_layerwise=false、use_gpu_connector_v3=true、save_only_first_rank=false。

路径	第一次 elapsed	第二次 full-hit elapsed	LMCache load	判断
干净基线：同一修复镜像，所有 CSA/HCA prefetch env 显式关闭	42.613 s	40.744 s	29,952 tokens；每 rank 13.26 GB，约 1.59-1.88 s	当前可比的正常 DSv4 + LMCache SSD-only 性能
CSA reuse seed 开启、decode prefetch 关闭： LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0	38.397 s	36.606 s	29,440 tokens；每 rank 13.03 GB，约 1.52-1.84 s	prefill reuse seed 不会拖慢正常 decode
CSA/HCA Python prototype 全开，decode prefetch/correction 每 token 执行	42.843 s	147.380 s	29,952 tokens；每 rank 13.26 GB，约 1.43-1.78 s	性能不可用；慢点是 decode 期 Python prototype
旧镜像 lmcache/vllm-openai:indexer-ssd	无有效数据	无有效数据	store 时崩溃	非 contiguous view 错误，不可作为 DSv4 baseline

结论：LMCache SSD full-hit load 本身约 1.5-1.9 s，不是 147 s 的瓶颈。当前 prototype 全开时，吞吐被每 token 每 CSA 层的 fire_async/correct_true_topk Python 路径拖垮。因此后续性能实验默认保持 LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0，先验证 LMCache full-hit prefill reuse 时的 seed/调度；decode 期 CSA prefetch 只在 accuracy 或 state-machine 实验中打开。

decode-gate 验证时，日志出现 LMCache hit tokens: 29440, need to load: 29440、每 rank retrieve 约 1.52-1.84 s、reuse prefetch seeded 30 CSA layers，且生成吞吐回到 约 14.8 tok/s。中间 121.010 s 的样本是部署错误导致旧代码仍在跑，不作为结果。

2026-05-28 prefill/decode 分拆：

测项	max_tokens	第一次 miss/store	第二次 full-hit/retrieve	说明
prefill/TTFT 稳定样本	1	TTFT 3.532 s	TTFT 1.873 s	两次之间等待 10 s
decode tail	512	TTFT 3.523 s；tail 34.580 s；约 14.78 tok/s	TTFT 1.867 s；tail 34.448 s；约 14.83 tok/s	decode prefetch 关闭，正常 DSv4 decode

full-hit 日志显示 `LMCache hit tokens: 29440, need to load: 29440`，每 rank retrieve 约 13.03 GB、1.42-1.73 s，并出现 `reuse prefetch seeded 30 CSA layers`。同时 `event=fire_async` 和 `event=correct_true_topk` 计数为 0，说明 decode 阶段没有跑 CSA Python prefetch/correction。若第一遍 store 完立刻第二遍 retrieve，曾出现 `partial retrieve / KV load failure`；稳定 prefill full-hit 测试需要等待数秒或使用已稳定存在的缓存 key。

2026-05-27 容器 `dsv4-indexer-ssd` 的 smoke 结果：

请求	prompt tokens	completion tokens	结果	关键日志
第一次同 prompt	17,875	1	2.600 s	store path 成功
第二次同 prompt	17,875	1	1.418 s	<code>Retrieved 16384 out of 16384 required tokens, reuse prefetch seeded 30 CSA layers</code>
LMCache hit 后 decode	17,875	16	7.121 s	<code>fire_async</code> 与 <code>correct_true_topk</code> 均执行

旁路性能采样：约 22,105 prompt tokens，`max_tokens=16`，命中 16,384 tokens 后约 4.354 s。日志聚合显示 `fire_async` median 约 1.5 ms，`correct_true_topk` median 约 4.8 ms，p95 约 29 ms，`read_ms` median 约 0.017 ms。当前慢点主要在 Python prototype correction/诊断路径，不在 SSD retrieve 本身。

旧资料使用顺序

优先级：

- 1. 本文。
- 2. `F:\LMCache\docs\design\v1\dsv4_csa_hca_prefetch_runbook.md`。
- 3. Claude memory 中的原始实验记录。
- 4. 旧 handoff。

旧 handoff 中出现冲突时，以本文为准。尤其注意：

- 不要把 DSv4 过滤成 61 个 KV cache 作为最终方案。
- 不要把 pool-only scoring 当成 production correctness path。
- 不要把 V2 residual proxy 写法用于 V4。
- 不要默认开启 prefill eviction 或 prefill proxy。
- 不要把不改变 HCA/CSA prefetch 语义的旁路优化写成当前主线。

2026-06-03 CSA prefill-only proxy 修正实现

本节修正此前错误实现：CSA prefill proxy 不能依赖 `LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=1`，不能走 `evict_after_prefill`，也不能把 `PREFETCH_PREFILL_ROWS=N` 实现成 多行 `N * topK` 的 flatten/union。正确语义是：LMCache prefix/full-hit 后，每个 TP worker 先 seed 自己本地的 CSA SSD/HBM pool；prefill 阶段只用最后一个 token row 的 CSA 预测结果发异步 SSD prefetch；官方 SparseAttnIndexer 产出的 true topK 仍然是 correctness source，并在同一 prefill 阶段做 correction。

本次修正后的 runtime 开关：

```
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_PREFILL_EVICTION=0
LMCACHE_INDEXER_PREFETCH_PREFILL_ROWS=1
LMCACHE_INDEXER_PREFILL_OVERLAP_PROFILE=0
LMCACHE_HCA_ENABLE_PREFETCH=0
```

关键实现点：

1. LMCacheConnectorV1Impl._maybe_seed_indexer_reuse_prefetch() 不再用 local_worker_id == 0 gate。DSv4 TP=8 时，每个 worker 都有自己的 CSA manager、pool 和 cursor，因此每个 rank 都必须 seed；否则 7 个 rank 会一直 cursor=0。
2. IndexerSSDManager.fire_async_for_layer() 中，proxy_rows > 1 识别为 prefill proxy。prefill proxy 只受 LMCACHE_INDEXER_PREFETCH_PREFILL_ROWS=1 控制，不受 DECODE_PREFETCH 控制。decode proxy 仍由 DECODE_PREFETCH 控制。
3. DeepSeek V4 hc_pre 的 residual 输入契约是 (... , hc_mult, hidden)，不能把 prefill residual 错压成 [1, hidden]。同时，当前 vLLM SparseAttnIndexer CUDA path 在 prefill 中绑定整段 prefill metadata，直接只传 1 行 proxy 会触发 cu_seq_len_k_start.size(0) == seq_len 断言。因此当前可运行 实现保持当前 prefill chunk 的 compute shape，例如 (503, 4, 7168)，但只从 topk_indices_buffer 取最后 1 行 topK 发 prefetch。这满足 PREFETCH_PREFILL_ROWS=1 的选择语义，不再做多行 union。
4. record_attention_topk_slots() 在 prefill proxy enabled 且 layer 已初始化时，由官方 SparseAttnIndexer 的 true topK 触发 prefill_correct_true_topk，补齐 proxy 没覆盖的真实 topK。

最小 smoke 验证：

```
host: gpu002
container: dsv4-256k-measure
model: deepseek-v4-pro
artifact: F:\LMCache\tmp_gpu002_logs\csa_prefill_proxy_smoke_20260603_v3.log
prompt: "The quick brown fox jumped over the lazy dog near the river bank. " * 1200
max_tokens: 1
```

结果：

request	status	elapsed	prompt tokens	说明
cold prefix	200	10.943 s	16,835	store/prefill，无 LMCache hit
prefix + tail hit	200	1.749 s	17,143	Retrieved 16640 out of 16640，External prefix hit rate 49.0%

日志计数：

event / marker	count	判断
reuse prefetch seeded	8	8 个 TP worker 都完成 CSA reuse seed
reuse_prefetch_seed	720	30 CSA layers/rank 的 submit + complete/timing 日志
using_tail_rows=1 / compute_shape=	240	30 CSA layers * 8 rank，prefill proxy 触发
event=prefill_fire_async	480	prefill 阶段真实 fire，不再只是 skip
prefill_correct_true_topk	240	官方 true topK 后 correction 触发
LMCacheRetrieveProfile / Retrieved	8 / 8	每 rank 成功 full-hit retrieve
RuntimeError / AssertionError / illegal memory	0 / 0 / 0	smoke 无崩溃

样例日志：

```
IndexerSSDManager: reuse prefetch seeded 30 CSA layers for request ... lmcache_tokens=16640
compressed_tokens~=4160
IndexerSSDManager: residual_proxy prefill layer 2 using_tail_rows=1 original_shape=(503, 4, 7168)
compute_shape=(503, 4, 7168)
IndexerSSDTiming: event=prefill_fire_async layer=2 total_ms=15.531 proxy_ms=15.270 filter_ms=0.092
submit_ms=0.000 prev=899 missing=0
IndexerSSDTiming: event=prefill_correct_true_topk layer=2 total_ms=2.476 drain_ms=0.066 miss_ms=0.077
read_ms=0.000 insert_ms=0.000 true=1024 missing=0
```

注意：当前实现是“语义正确、可运行”的 prefill-only proxy 版本；它为了适配 vLLM prefill metadata，proxy compute 仍按当前 chunk 执行，只在结果选择上限定最后 1 行。后续若要进一步优化 proxy compute cost，需要为 SparseAttnIndexer 增加不依赖全局 prefill ForwardContext 的单行/局部 metadata path，而不是重新打开 DECODE_PREFETCH 或 evict_after_prefill。

运行环境同上一节 corrected prefill-only proxy，仍保持：

```
LMCACHE_INDEXER_ENABLE_REUSE_PREFETCH=1
LMCACHE_INDEXER_ENABLE_DECODE_PREFETCH=0
LMCACHE_INDEXER_ENABLE_PREFILL_EVICTION=0
LMCACHE_INDEXER_PREFETCH_PREFILL_ROWS=1
LMCACHE_HCA_ENABLE_PREFETCH=0
```

请求：

request	status	elapsed	prompt tokens
cold/store prefix	200	47.597 s	248,452
prefix + tail hit	200	6.252 s	248,754

LMCache load 证据：

```
Total tokens 248754, Inference Engine computed tokens: 0,
LMCache hit tokens: 248320, need to load: 248320
Retrieved 248320 out of 248320 required tokens (from 248320 total tokens).
```

因此这次 full-hit 的实际 load 长度是 **248,320 tokens**，不是 248,754 total prompt tokens；tail 仍需现算。每 rank retrieve size 是 **4.7469 GB**，8 rank 合计约 **37.98 GB**。8 rank retrieve profile：

metric	mean	min	max
LMCache retrieve total_ms	1378.135 ms	1329.160 ms	1445.565 ms
process_tokens_ms	408.731 ms	362.936 ms	470.671 ms
to_gpu_ms	966.843 ms	952.381 ms	983.591 ms

hit-only CSA timing：

event	count	mean total	median total	sum total	关键分项
reuse_prefetch_seed	240	78.752 ms	60.213 ms	18.900 s	read_ms sum 12.427 s, load_ms sum 5.165 s
prefill_fire_async	480	19.271 ms	5.001 ms	9.250 s	proxy_ms sum 4.485 s, submit_ms sum 4.701 s
prefill_correct_true_topk	240	30.475 ms	29.037 ms	7.314 s	drain_ms sum 5.000 s, read_ms sum 0.063 s, insert_ms sum 2.010 s

wall-clock 顺序：

marker	first	last	span
LMCache hit tokens: 248320	07:04:06.624	-	-
LMCacheRetrieveProfile	07:04:07.993	07:04:08.111	118 ms
reuse_prefetch_seed timing	07:04:08.103	07:04:09.717	1.614 s
prefill_fire_async	07:04:09.728	07:04:12.298	2.570 s
prefill_correct_true_topk	07:04:09.792	07:04:12.332	2.540 s

判断：

1. LMCache load 是 full-hit： **248320 / 248320 required tokens**。
2. corrected prefill proxy 确实触发，但 248K hit elapsed 变成 6.252 s，明显慢于无 prefill proxy 时约 2-3 s 的 full-hit TTFT。
3. proxy 发出的 SSD read 本身基本被掩住：**prefill_correct_true_topk.read_ms** hit-only sum 只有 62.714 ms，mean 0.261 ms/event。
4. 没被掩住的是 Python/prototype 路径和写入/等待开销：**prefill_fire_async.proxy_ms**

- `submit_ms`、`prefill_correct_true_topk.drain_ms` + `insert_ms` 贡献最大。

5. 因此当前 corrected prototype 不能作为性能收益证据；它证明了 prefill-stage CSA proxy/correction 语义可运行，也证明 SSD read wait 已大体隐藏，但实现成本 主要转移到了 proxy compute、future submit/drain 和 HBM pool insert。